

Vibe Coding 실무 과정

위블 CEO 노규남
bardroh@weable.ai

2026/06/16

- 클라우드 업체(가비아)와 회선 업체(케이아이엔엑스) CTO 역임
- 현재는 인공지능 스타트업인 위블(<https://www.weable.ai>) 의 CEO로 재직 중: 머신러닝과 클라우드, DevOps 등에 대한 **컨설팅** 및 **솔루션 개발**을 영역으로 비즈니스를 전개, **교육사업**도 진행
- AI를 중심으로 운영과 관리에 클라우드 기술을 접목하려고 시도하고 있음(MLOps, AIOps)
- AI: GBM 등 Classical Machine Learning부터 CNN, RNN, Transformer 기반의 NLP, DQN, CFR, LLM, 모방학습까지 머신러닝 전반에 걸친 솔루션 개발 및 컨설팅, 교육
- 클라우드: Public, Private, Hybrid IaaS, docker, kubernetes, MSA(MicroService Architecture), CI/CD 등 클라우드 네이티브한 영역에서의 개발 및 운영 역량 보유
- AI 중에서도 1) sLLM 2) 온디바이스 AI 3) 로봇 AI가 회사에서 가장 관심있게 연구하고 있는 영역임

검색 증강 챗봇

선택 업로드

검색에 사용할 파일을 선택해주세요.

(교육교재) 삼성전자 CRA 과... x v

파일 삭제

클라이언트 쪽의 인터페이스를 구현해야 한다. 이렇게 하면 코드가 더 간결해서 유지보수가 용이해집니다.

5. Dependency Inversion Principle (의존성 역전 원칙): 고수준 모듈은 저수준 모듈에 의존해서는 안 된다. 즉, 추상화된 인터페이스에 의존하게 되어야 한다. 이렇게 설계되면 코드가 더 유연해져 확장성이 좋아집니다.



리스코프 치환 원칙에 대해서 더 상세하게 설명해주세요.



리스코프 치환 원칙(Liskov Substitution Principle, LSP)은 객체지향 프로그래밍에서 중요한 원칙 중 하나입니다. 이 원칙은 상위 타입의 인스턴스를 하위 타입의 인스턴스로 교체할 수 있는지 여부에 대한 규칙을 제공합니다. 즉, 프로그램의 다른 부분이 상위 타입으로 선언된 변수에 하위 타입의 객체를 할당할 수 있는지 확인하는 것입니다.

리스코프 치환 원칙은 다음과 같은 조건을 만족해야 합니다:

1. 상위 타입의 객체와 하위 타입의 객체가 서로 대체 가능해야 한다. 이렇게 하면, 프로그램의 다른 부분은 상위 타입으로 선언된 변수에 상위 타입의 객체를 할당하더라도, 실제로 하위 타입의 객체가 할당되어 있는 경우에도 정상적으로 작동해야 합니다.
2. 하위 타입의 객체가 상위 타입의 인터페이스를 구현해야 한다. 이렇게 하면, 상위 타입의 변수에 하위 타입의 객체를 할당하더라도, 상위 타입의 메서드와 프로퍼티를 사용할 수 있습니다.
3. 하위 타입의 객체가 상위 타입의 인터페이스를 구현할 때, 상위 타입과 동일한 방식으로 작동해야 합니다. 즉, 상위 타입과 하위 타입 간에는 일관성이 있어야 합니다.

리스코프 치환 원칙은 코드의 유연성과 재사용성을 높여주며, 객체지향 설계를 더 안정적으로 만들어줍니다. 이 원칙을 따르면, 프로그램의 다른 부분에서 상위 타입의 변수에 하위 타입의 객체를 할당하더라도, 프로그램이 정상적으로 작동하게 됩니다. 이렇게 함으로써, 코드가 더 유연하고 재사용 가능해지며, 설계의 안정성이 향상됩니다.

대화를 입력해주세요.



NL2SQL

회사의 데이터에 대해서 자연어로 질의하 시면 해당하는 BigQuery 쿼리를 생성합니다.

상세하게 질의하실 수록 정확한 답을 얻을 수 있습니다.

데이터 카탈로그는 매일 업데이트 됩니다.



2023년 9월 1일부터 2023년 9월 30일까지 '닥터픽스' 브랜드의 재고 총합을 구해줘.



```
SELECT SUM(quantity) AS total_stock
FROM company_in.daily_stock_shop_stock
WHERE date_logged BETWEEN '2023-09-01' AND '2023-09-30'
AND brand = '닥터픽스';
```



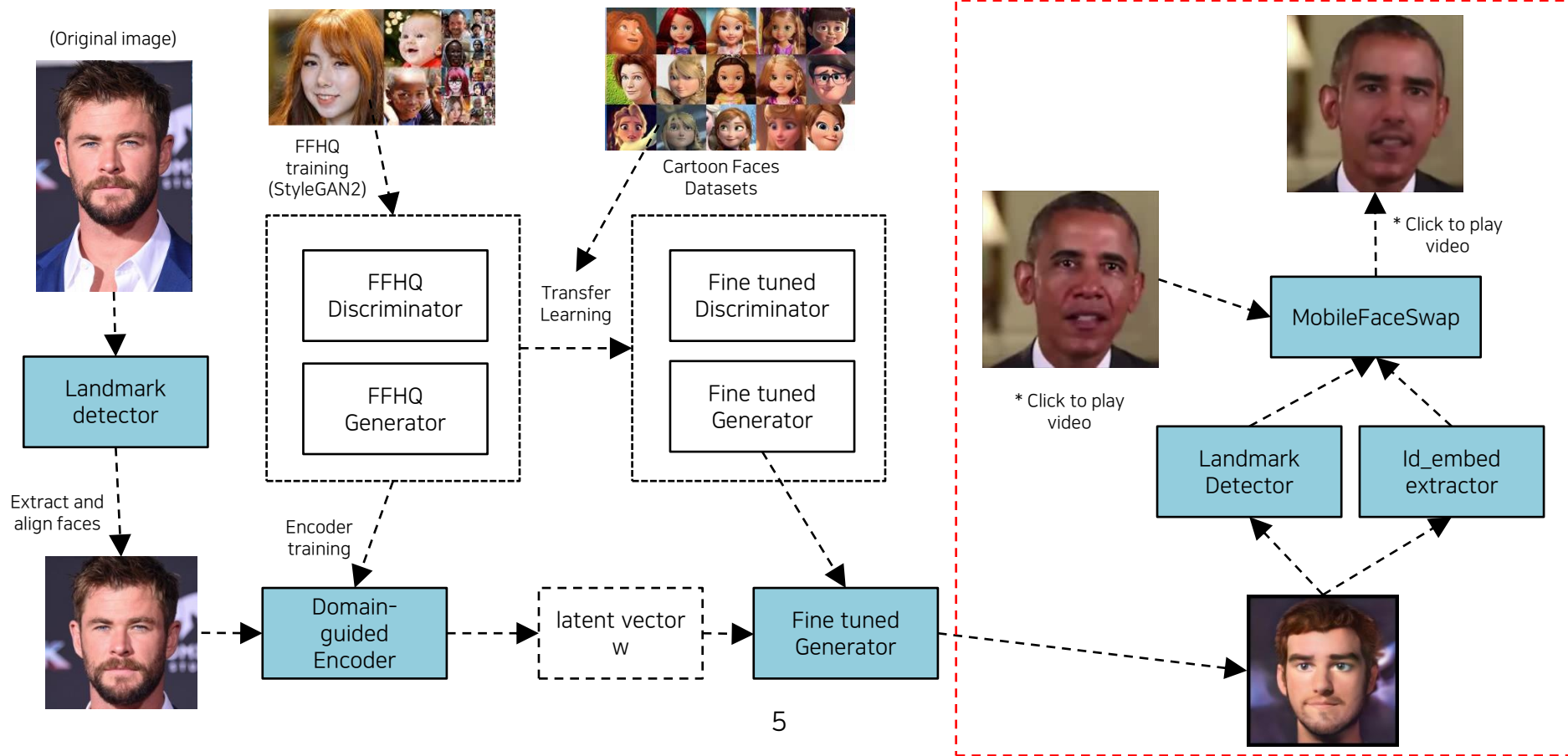
2023년 데이터로부터 전체 브랜드의 월별 재고 평균을 구해줘.

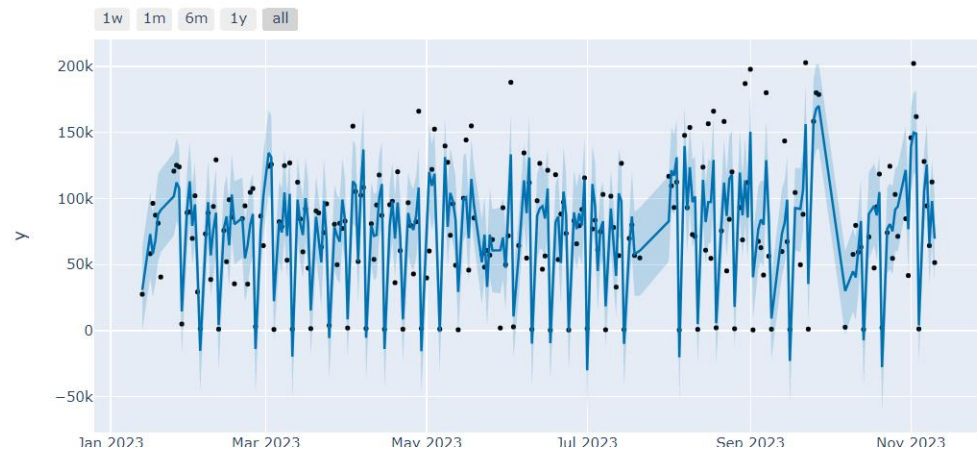
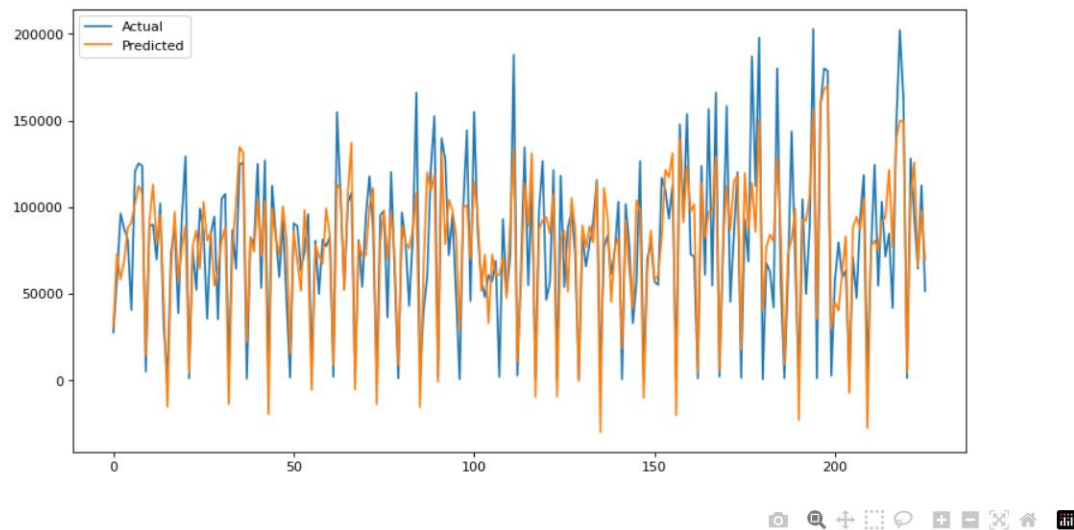


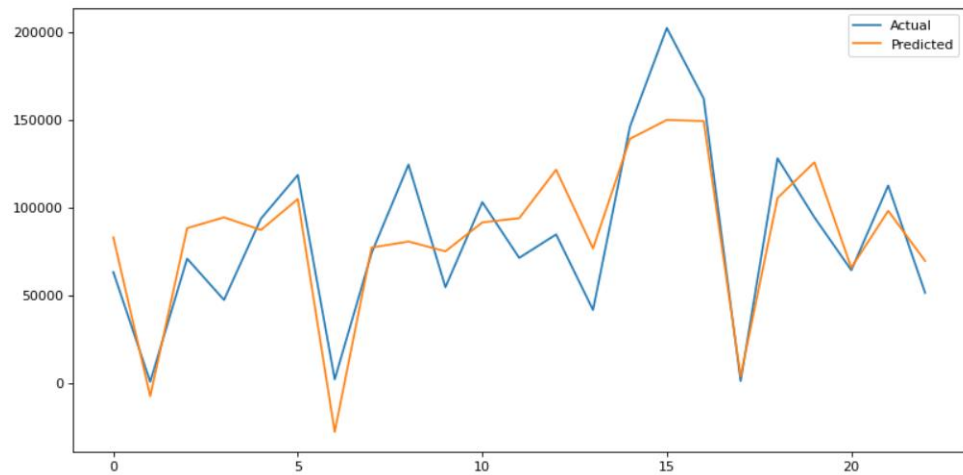
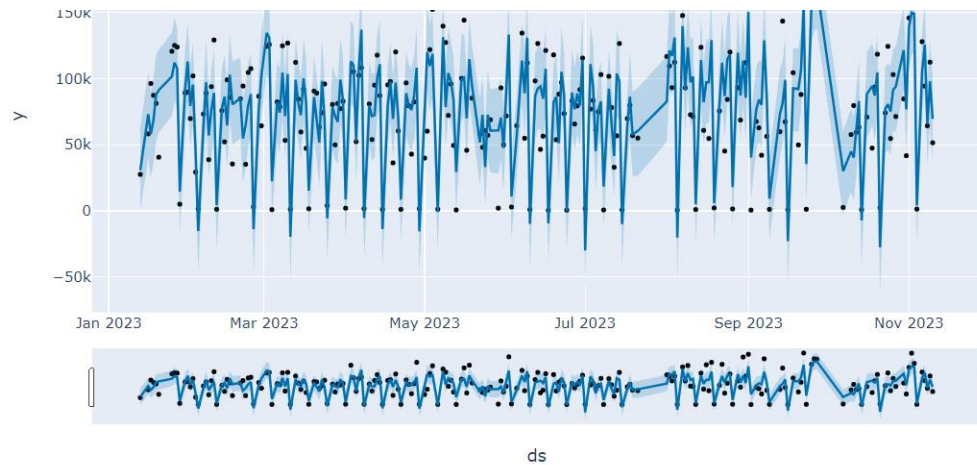
```
SELECT
  EXTRACT(YEAR FROM date_logged) AS year,
  EXTRACT(MONTH FROM date_logged) AS month,
  brand,
  AVG(quantity) AS average_stock
FROM
  company_in.daily_stock_shop_stock
WHERE
  EXTRACT(YEAR FROM date_logged) = 2023
GROUP BY
  year, month, brand
ORDER BY
  year, month;
```

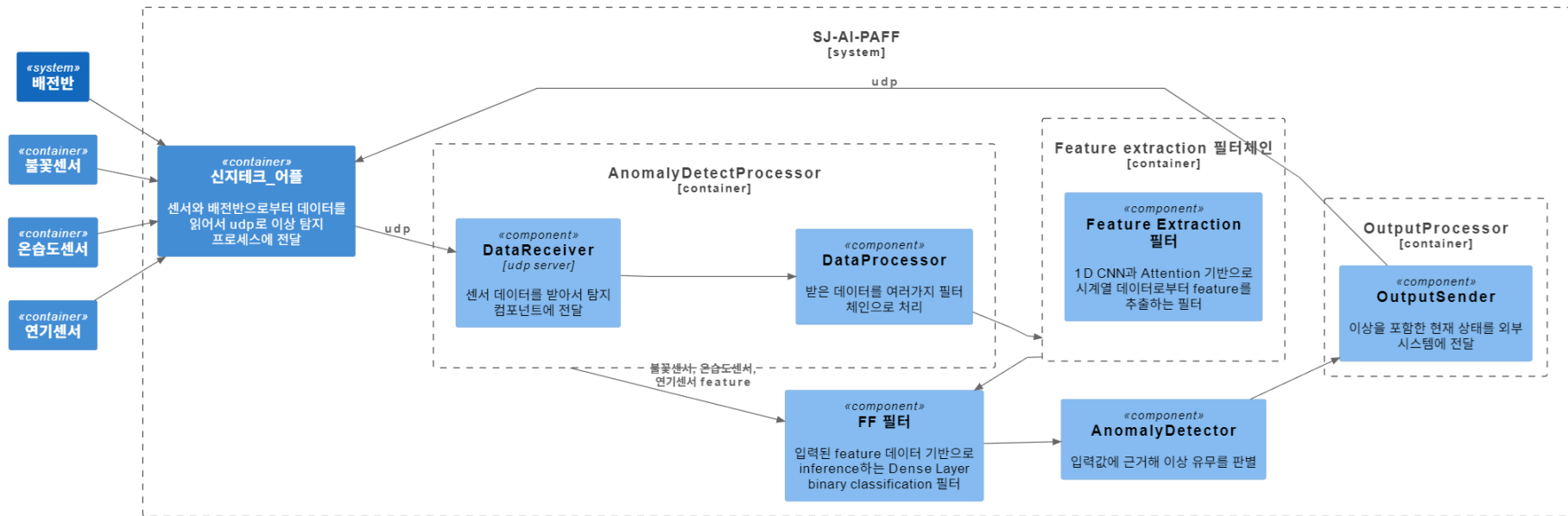
대화를 입력해주세요.











하드웨어 박스 기반 로컬 LLM 솔루션

- 공유 메모리 128G 구성으로 최대 96GB VRAM 할당, 하드웨어 구성은 변경 가능
- 고객의 용도에 맞춰서 커스터마이징 가능한 sLLM 구동을 위한 하드웨어 박스: HW와 SW 모두 최적화
- 일반적인 챗봇, 다양한 종류의 RAG, 워크플로우 자동화, 이미지 및 동영상 생성을 하나의 박스에서 모두 처리할 수 있으므로 보안적인 측면에서의 문제가 원천적으로 해소
- 추가적인 비용 없이 무제한으로 사용할 수 있으며, 사용량 증가에 따라 유연하게 확장



Attention Mechanism Improve

localhost:8080/c/5b217ee4-1233-4cc9-860b-4b9b25f45ecd

Open WebUI

새 채팅

검색

노트

워크스페이스

채팅

오늘

Attention Mechanism Improve

Hello World

Korean Greeting

새 채팅

Greeting & How Are You?

어제

Hello Korean Conversation

노규남

실제 Attention 계산하는 공식을 설명해줘

AI

w4 Today at 오후 2:38

Attention 메커니즘은 입력 시퀀스에서 각 위치의 중요도를 계산하여 가중치를 부여하는 방식으로 동작합니다. 가장 기본적인 형태의 Attention 계산 공식은 다음과 같이 설명할 수 있습니다.

Scaled Dot-Product Attention 공식

Transformer 모델에서 사용되는 Scaled Dot-Product Attention의 계산 과정은 다음과 같습니다:

입력

- **Query (Q): **Q**: 입력 벡터에서 '질문'을 나타내는 벡터로, shape는 (seq_len, d_k) .
- **Key (K): **K**: 입력 벡터에서 '키'를 나타내는 벡터로, shape는 $((seq_len, d_k)$.
- **Value (V): **V**: 입력 벡터에서 '값'을 나타내는 벡터로, shape는 $((seq_len, d_v)$.

단계별 계산

1. Dot-Product: Query와 Key 벡터의 내적을 계산합니다.

$$scores = Q \cdot K^T \quad (\text{shape: } (seq_len, seq_len))$$

2. Scaling: 내적 결과의 값을 $\sqrt{d_k}$ 로 나눕니다. 이는 다음과 같이 표현됩니다:

$$scaled_scores = \frac{scores}{\sqrt{d_k}}$$

3. Softmax (정규화): Scaled scores에 Softmax 함수를 적용하여 각 위치의 가중치를 계산합니다.

$$attention_weights = softmax(scaled_scores)$$

여기서 Softmax 함수는 각 열(column)마다 적용합니다.

4. Attention 계산: Obtained weights를 Value 벡터에 곱해 ↓ 최종 Attention 출력을 만듭니다.

메시지 보내기

+ 코드 인터프리터

Workflow Editor

Free AI Image Generator - n8n

Overview

Editor

Executions

Evaluations

Inactive

Share

Save

Star

134,639

When chat message received

Fields - Set Values

AI Agent - Create Image...

Code - Clean Json

Code - Get Prompt

Code - Set Filename

HTTP Request - Create Image

Save Image To Disk

Chat Model Memory

Tool Output Parser

Model

OpenAI Chat Model

Chat

Session: 73494...

Logs

HTTP Request - Create Image Success in 16.992s

모퉁 부위가 비어 있고 그 안에서 촛불이 타고 있는 여성을 그려줘. 촛불은 노란 불꽃을 내면서 타고 있어. 여성은 젊고 금발에 머리가 길며 신비로운 분위기로 중세풍 드레스를 입고 웃고 있어. 배경은 낮의 마을이야. 최대한 사실적인 이미지로 그려주고 여성의 몸 전체가 나오도록 그려줘.

```
{ "body": { "prompt": ""Medieval village setting | Young woman with long blonde hair, wearing a mystical medieval dress, smiling, with a hollow torso containing a burning candle emitting a yellow flame | text elements: 'Luminary' - elegant calligraphy, integrated naturally into the dress's fabric ::7 | Cinematic dramatic lighting, natural daylight, soft shadows, depth of field blur, clean and slightly rustic environment ::7 | 8k resolution, hyperrealistic, photorealistic quality, octane render, cinematic composition --ar 16:9 --s 1000 --q 2 --chaos 20 --styleize 400"", "image_size": { "width": "1080", "height": "1920" }, "num_inference_steps": 12, "guidance_scale": 3.5, "num_images": 1, "enable_safety_checker": true }, "fileName": "c:\\work\\images\\{{{ $('Code').item.json.fileName }}}"} }
```

OUTPUT

← Back to list



Templates

Variables

Insights

Help

What's New

Kyounam Roh

SO ARM-101: 6자유도(6-DoF) 로봇팔

- Leader Arm과 Follower Arm간의 연동을 통한 Teleoperation
- 연결된 카메라와 모터의 센서를 통한 멀티 모달 데이터 수집 가능: 영상과 센서 데이터를 통합하여 학습
- Zero-shot Generalization: 사전 학습된 정책을 활용해 새로운 작업에 즉시 적응
- Behavioral Cloning 알고리즘을 적용하여 Transformer 기반 정책 네트워크(Action-Chunking-Transformer)와 Diffusion Policy를 통해 연속적인 동작을 구현



Weable AI Grader

ai-grader.weable.ai/admin/rubrics/rbc_3d27bec2

Weable

AI Grader

메뉴

대시보드

과정

과제 (루브릭)

채점 결과

weable72

admin

로그아웃

과제 (루브릭) 편집

과제 (루브릭) YAML 편집 및 생성

< 과제 (루브릭) 목록 >

day01/task01 피보나치 수열

과제 텍스트

day01/task01 과제 - 피보나치 수열 (에이전트 채점 모드 E2E)

정수 n이 주어졌을 때 피보나치 수열의 n번째 값을 계산하는 프로그램을 작성하세요.

요구사항:

1. fib(n) 함수를 정의하세요 (n=0이면 0, n=1이면 1).

제목

day01/task01 피보나치 수열

채점 방식

☒ 표준 채점

☐ LLM 리뷰

☐ 에이전트

전체 자동생성

요구사항만 추출

루브릭 YAML

교육생용 요약

요구사항

meta:

id: day01_task01_fibonacci_agent

title: day01/task01 피보나치 수열

language: ko

description: 정수 n에 대한 피보나치 수열 값을 계산하고 fib(10)의 결과를 출력하는 과제

grading_mode: agent

agent_config: null

submission:

required_files:

- app.py

criteria:

pass_score: 60

items:

- id: req_1

title: fib(n) 함수 정의

score: 40

저장

초기화

과제 ID

rbc_3d27bec2

도움말

전체 자동생성

과제 텍스트 → YAML + 요한 한 번에 생성

요구사항만 추출

과제에서 요건 목록만 추출 후 확인·수정

마크다운-YAML 재생성

확인한 요구사항 기반으로 YAML 재생성

- 이 과정의 주제는 'AI 코딩 에이전트를 이용한 개발'로, LLM을 개발에 적극적으로 이용하여 효율을 올릴 수 있는 방법과, 사용되는 도구, 워크플로우 등에 대해서 알아보고, 실습을 통해 이런 내용들을 숙지하는 것
- 교육시간은 화~금요일 오전 9시~오후 6시까지, 일일 8시간, 전체 32시간 분량
- 매 시간 정각에 시작하며, 50분 수업 후 10분 휴식을 반복. 점심시간은 오후 12시~1시
- 수강생들 모두가 포함된 git repo(github)를 공유: 메일로 초청하여 멤버가 되며 이를 통해 필요한 파일들을 전달하고 실습, 과정 이해도를 간접적으로 평가
- 매일의 take-away가 있으며, 매일의 실습이 있음: 이전 과정의 내용을 기반으로 다음 과정을 진행하는 것이므로 실습을 꾸준히 따라오는 것이 중요

- AI 코딩은 인공지능을 활용한 모든 코딩 방식을 아우르는 포괄적 개념
- GitHub Copilot, ChatGPT, Claude 등의 AI 도구를 활용해 코드 자동 완성, 버그 탐지, 코드 설명 등을 지원
- AI는 코드의 일부를 예측, 생성하거나 블록 단위로 분석하는 방식으로 개발자를 보조하며, 주도권은 개발자에게 있음
- 바이브 코딩은 Andrej Karpathy가 2025년 2월 3일 X(트위터)에서 처음 제시한 개념
- 개발자는 직관과 큰 그림을 담당하고, 구체적인 구현은 AI가 맡는 협업 방식
- 코드의 작동 원리를 완전히 이해하지 않아도 개발이 가능하지만, '기술을 전혀 몰라도 개발할 수 있다'는 다소 과장에 가까움: 기술을 알아야 이후의 검증을 직접 하거나, 또는 검증 조건을 규정할 수 있음
- 이번 강의에서는 AI 코딩 방법론 전반에 대해서 설명하며, 주로 Claude Code를 중점적으로 다룰 예정: Claude Code가 이 업계 표준 도구에 가까움

- 주요 도구: GitHub Copilot, Cursor, Replit, Windsurf, Claude Code, Gemini CLI
- 실리콘밸리를 중심으로 빠르게 확산 중이며, 국내에서도 점차 도입이 늘고 있음
- 현실적 제약
 - AI 시대에도 코드를 읽고 이해하는 능력은 여전히 핵심 역량
 - AI 도구는 초심자보다 숙련된 개발자에게 더 큰 생산성 향상을 제공
 - AI가 버그를 해결하지 못할 경우, 반복적인 수정 시도가 필요하며 오히려 시간이 더 걸릴 수 있음
 - 코드 품질 검토나 보안 이슈는 개발자가 직접 확인하는 것이 중요: 검증 이슈
- 미래 전망
 - 바이브 코딩은 개발 패러다임의 변화를 이끌고 있으며, 숙련된 개발자의 판단력과 AI의 생산성이 결합된 협업 형태가 주류가 될 것으로 전망

바이브 코딩 도구 비교표

제품	무료 범위	유료 플랜 및 가격	주요 특징 및 비교
GitHub Copilot	월 2,000회 코드 추천 + 채팅 제한, IDE 연동 (Free 플랜)	Pro: 월 \$10, Pro+: 월 \$39, Business: 월 \$19/사용자. 2026.6월부터 AI 크레딧 기반 사용량제로 전환 예정	주요 IDE 지원, 자동완성·AI 채팅 포함. GPT-5, Gemini 3 등 멀티모델 지원
Codex	ChatGPT Plus/Pro 플랜 포함 제공	Plus: 월 \$20, Pro: 월 \$200	GPT 계열 코드 모델 기반. 데스크탑·CLI·IDE 확장·웹 브라우저 모두 지원. 커스텀 에이전트 & API 워크플로우에 적합
Claude Code	없음(Trial만 제공)	Pro: 월 \$20, Max: 월 \$100~\$200, Enterprise: 연 \$50,000~	Claude Opus / Sonnet / Haiku 사용 가능, 터미널 기반 AI 코딩 에이전트, 심층 인텍싱 시 토큰 소모 큼
Windsurf (구 Codeium)	무료 플랜: 탭 자동완성 무제한 Cascade 세션 제한적 제공	Pro: 월 \$20, Max: 월 \$200, Teams: 월 \$40/사용자. 2026.3월 크레딧 -> 쿼터제로 변경, 가격 인상	Cascade 에이전트, 코드베이스 전체 컨텍스트 이해
Open Code	완전 무료 (오픈소스), 자체 LLM API 키 필요	Go 플랜: 월 \$10 (저비용 모델 접근) Zen 플랜: API 원가 그대로 과금	오픈소스 터미널 기반 Cursor 대안. Ollama 로컬 모델 또는 외부 LLM 연동, LSP(언어 서버 프로토콜) 내장, 오프라인 동작 가능, MCP 서버 통합 지원
Cursor	무료: 약 2,000건 자동완성 + 50회 GPT-4/Claude 요청	Pro: 월 \$20 (500회 요청), Business: 월 \$40	VS Code 기반 독립 IDE, 코드베이스 채팅·리팩토링 지원. Claude Sonnet / Opus, GPT-5 등 멀티모델 선택 가능. 200만 + 사용자
Amazon Q Developer (구 CodeWhisperer)	개인 무료(거의 무제한), AWS ID 필요	Pro: 월 \$19/사용자	2024.4월 CodeWhisperer에서 리브랜딩. 인프라 코드(IaC) 자동 추천, 보안 취약점 자동 수정, 에이전트 기능(기능 구현·문서화·리팩토링) 추가
Gemini CLI	무료: 1,000 req/일 (개인 Google 계정) 일일 제한 자동 갱신	Pro: 월 \$19.99 기업 플랜: Google Cloud 콘솔 별도	로컬 CLI 기반 오픈소스, Gemini 3.1 모델, 다양한 언어·문맥 지원. GEMINI.md로 프로젝트 지시사항 지속 유지, MCP 서버 연동 가능
Replit Ghostwriter	Replit Core 플랜으로 일부 기능 제공, AI 추천 제한	Core: 월 \$25 (AI 포함), 연 \$15/월	웹 IDE 기반, 채팅 내 오류 자동 수정, 웹/모바일 지원. 에이전트가 프로덕션 DB 삭제 사례 등 주의 필요
Antigravity (CLI)	사용량 제한 있는 무료 (베타)	Pro/Team: 협의 기반 가격	에이전트 중심, 브라우저 서버에이전트로 UI 테스트 자동화, 스크린샷·녹화로 시각적 결과 검증. Cursor 유료 플랜 수준의 기능 제공

- Claude Code는 윈도우, 리눅스, MacOS 등 어떤 환경에서도 설치 가능: VS Code 기반의 어플리케이션
- 다만 서비스에는 리눅스 또는 Linux-like한 환경이 좀더 적합: 오픈소스 패키지들이 대부분 리눅스나 MacOS 기반으로 개발되고 있고, 서버는 거의 리눅스 환경에서 구동
- 이 과정에서는 로컬 PC에 wsl을 설치하고, wsl에서 Claude Code를 연동해서 사용할 예정
- WSL2는 Windows Subsystem for Linux2로, 윈도우에서 리눅스 어플리케이션을 구동하게 해주는(정확하게 말하면 Linux system call을 처리해주는) 호환성 레이어로, 리눅스 커널의 기능을 제공
- WSL2를 사용하면 윈도우안에서 리눅스 배포본을 구동할 수 있고, 리눅스 어플리케이션도 모두 실행할 수 있음: 거의 대부분의 어플리케이션을 무리 없이 실행 가능
- 가상화 기능이 있는 Windows 10 이상의 PC에서는 모두 사용 가능: Mac 사용자는 wsl을 사용할 필요가 없으며 자체적으로 git과 AI agent 도구들을 설치해서 사용하면 됨

- 제어판 > 프로그램 > 프로그램 및 기능 > Windows 기능 켜기 또는 끄기
- Linux용 윈도우 하위시스템 켜기, 가상 머신 플랫폼 켜기
- 재부팅 후 Microsoft Store에서 Ubuntu를 검색하고 22.04를 선택하여 설치
- cmd.exe 실행
- `wsl --install Ubuntu-22.04`로 설치, ubuntu 계정 생성하고 암호 설정
- `wsl -i -v`로 현재 설치된 배포본 확인
- `wsl -d Ubuntu-22.04`로 wsl 콘솔 진입
- 종료는 `wsl --terminate Ubuntu-22.04`
- 배포본 삭제는 `wsl --unregister Ubuntu-22.04`

1) github 계정 생성, wsl 로그인

2) 작업 디렉토리 생성

```
mkdir ~/work
```

```
cd ~/work
```

3) git 설정

```
git config --global user.name "Bard K. Roh"
```

```
git config --global user.email bardroh@weable.ai
```

```
git config --global pull.rebase false
```

```
git config --global --list
```

4) ssh키 설정 & git clone

```
ssh-keygen
```

```
cat ~/.ssh/id_rsa.pub
```

화면의 내용을 전부 복사해서 github의 Settings-SSH Keys에 붙임

```
git clone git@github.com:weable-kosa/kosa-vibecoding-2026-3rd.git
```

5) 간단한 파일을 만들어서 추가

```
cd ~/work/kosa-vibecoding-2026-3rd/src/exercise/{본인id}/day01
```

```
history > h1.txt
```

```
git add h1.txt
```

```
git commit -m 'add history'
```

```
git pull--no-edit
```

```
git push
```

ssh clone에서 timeout 날 경우에는 https clone을 시도해볼 수 있음

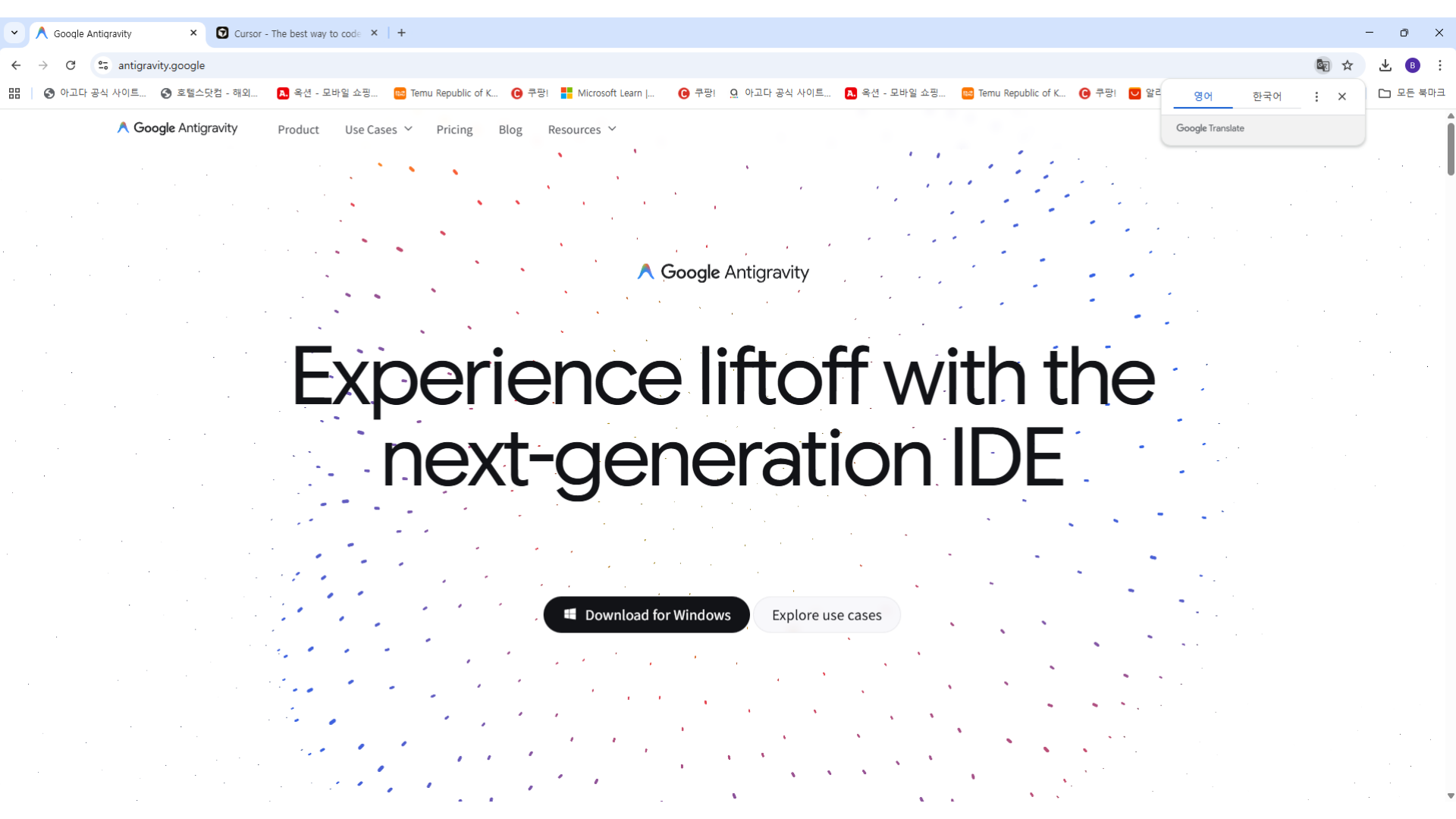
1) github token 설정

- github의 Settings-Developer Settings에서 Personal Access Token 선택
- Token(classic)을 새로 생성, repo 관련 모든 권한만 할당

2) 다음 명령으로 https token 기반 clone

```
git clone https://[사용자 id]:[access token]@github.com/weable-kosa/kosa-vibecoding-2026-3rd.git
```

- Antigravity는 AI 기반 개발 워크플로우를 지원하는 구글의 AI 코딩 어시스턴트
- Cursor와 마찬가지로 VS Code fork로 개발되었기 때문에, VS Code의 플러그인들을 거의 그대로 활용할 수 있음: WSL, python, jupyter 기타 등등
- 구글 계정만 있으면 사용 가능하며, 무료인 Individual plan부터 Developer plan, Team plan, Organization plan 등 다양한 요금제 옵션을 제공함
- Individual plan에서는 자동완성 무제한과 명령 무제한을 제공하며, Agent 구동에는 rate limit이 있으나 사용량은 무료치고 매우 넉넉한 편: rate limit은 주간으로 리셋된다고 함
- Fast와 Planning의 두가지 모드를 제공하고, Fast가 다른 서비스의 Agent + Ask, Planning이 Cursor의 Plan 모드에 해당함
- 다른 서비스 대비 실행이 다소 느리고, 복잡한 작업은 잘 못하는 경향이 있으나 무료로 사용할 수 있는 가장 좋은 코딩 에이전트 중 하나



Experience liftoff with the next-generation IDE

 Download for Windows

Explore use cases

FileEditSelectionViewGoRunTerminalHelp

Explorer

tetris [WSL: Ubuntu-22.04]

.cursor

backend

__pycache__

database.py

main.py

models.py

schemas.py

game.html

game.js

index.html

requirements.txt

style.css

tetris_scores.db

Outline

Timeline

style.css

```
{ } style.css > ...
1  * {
2      margin: 0;
3      padding: 0;
4      box-sizing: border-box;
5  }
6
7  body {
8      font-family: 'Roboto', 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
9      background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
10     min-height: 100vh;
11     display: flex;
12     justify-content: center;
13     align-items: center;
14     color: #333;
15 }
16
17 /* Material Design Typography */
18 .material-headline {
19     font-family: 'Roboto', sans-serif;
20     font-weight: 400;
21     font-size: 24px;
22     line-height: 32px;
23     letter-spacing: 0;
24 }
25
26 .material-body {
27     font-family: 'Roboto', sans-serif;
28     font-weight: 400;
29     font-size: 14px;
30     line-height: 20px;
31     letter-spacing: 0.25px;
32 }
33
34 /* Material Design Elevation */
35 .material-elevation-0 {
36     box-shadow: none;
37 }
38
39 .material-elevation-1 {
40     box-shadow: 0 2px 1px 1px #000, 0 0 0 2px #000, 0 1px 1px 0 #000, 0 0 0 1px #000;
41 }
```

Creating FastAPI Backend

Analyzed JS game.js#L1-392

> Thought for 11s

Edited Task

Edited Implementation Plan

Backend Setup

I have analyzed the existing frontend code and planned the backend structure. I created the task list and implementation plan. Next, I will start creating the backend files.

Progress Updates

Expand all

1 Preparing to set up backend files

I have created a plan to implement the FastAPI backend and integrate it with the frontend. Please review the implementation plan.

Implementation Plan

Open

작성된 계획대로 구현해줘

> Thought for 10s

_.cursor\tetris > mkdir -p backend

Ran terminal command Go to Terminal · Exit code 0

Always Proceed

Edited database.py +13 -0

Open diff

0 Files With Changes

Review Changes

Ask anything (Ctrl+L), @ to mention, / for workflows

+ ^ Fast ^ Gemini 3 Pro (Low)

Ln 34, Col 32 Spaces: 4 UTF-8 LF CSS AGQ Antigravity - Settings

- Gemini CLI는 터미널 환경에서 직접 AI와 상호작용하며 개발 워크플로우를 지원하는 구글의 명령줄 기반 AI 도구. Claude Code나 Codex와 비교됨
- 별도의 IDE(VS Code 등)를 실행하지 않고 터미널 내에서 직접 코드 생성, 버그 수정, 셸 명령어 추천 등을 수행 가능: AI agent 모드로, 대부분의 작업을 위임 받아서 구동하는데 최적화
- API Key 기반 또는 구글 계정 인증을 통해 사용하며, 터미널 환경의 가벼움과 속도를 유지하면서 Gemini 모델의 추론 능력을 활용함: 구글 계정만 있으면 바로 사용할 수 있음
- 무료 및 유료 플랜: 제한적으로 무료로 쓸 수도 있으며, Google AI Pro나 AI Ultra, Gemini Code Assist를 통해 유료 가입하고 quota와 기능을 확장할 수 있음. Flash, Flash-lite, Pro 사용량을 각각 집계하고 24시간 단위 quota
- 무료 플랜의 사용량이 가장 후하다는 평가: 품질은 보장하기 어려우나 무료임을 감안할 때 준수한 편. Antigravity와 사용량을 공유
- 2026년 6/18 이후 개인 고객은 사용이 중단되며 Antigravity CLI로 통합

```
Gemini - lerobot  x  +  v  -  □  x
ubuntu@DESKTOP-JJ3MK15:~/work/lerobot$ gemini

> GEMINI

Tips for getting started:
1. Ask questions, edit files, or run commands.
2. Be specific for the best results.
3. /help for more information.

> 이 repo에 있는 소스에 대해서 설명해줘

◆ 네, LeRobot 리포지토리의 소스 코드에 대해 설명해 드리겠습니다. 제공해주신 GEMINI.md 파일의 분석 내용을 바탕으로 정리했습니다.

LeRobot 프로젝트 개요

LeRobot은 실제 로봇 공학 연구 및 개발을 위한 허깅페이스(Hugging Face) 라이브러리입니다. 주요 목표는 다음과 같습니다.

* 진입 장벽 낮추기: 누구나 쉽게 로봇 공학 데이터셋과 사전 훈련된 모델을 공유하고 사용할 수 있도록 합니다.
* 핵심 기술 초점: 모방 학습(Imitation Learning)과 강화 학습(Reinforcement Learning)에 중점을 둡니다.
```

Ready (tetris)

Signed in with Google /auth
Plan: Gemini Code Assist for individuals /upgrade

> /model

Select Model

- 1. gemini-3-flash-preview
- 2. gemini-3.1-flash-lite-preview
- 3. gemini-2.5-flash
- 4. gemini-2.5-flash-lite

Remember model for future sessions: false (Press Tab to toggle)
> To use a specific Gemini model on startup, use the --model flag.

Model usage

Flash		8%	Resets: 12:05 PM (23h 36m)
Flash Lite		0%	Resets: 12:30 PM (24h 1m)
Pro		100%	

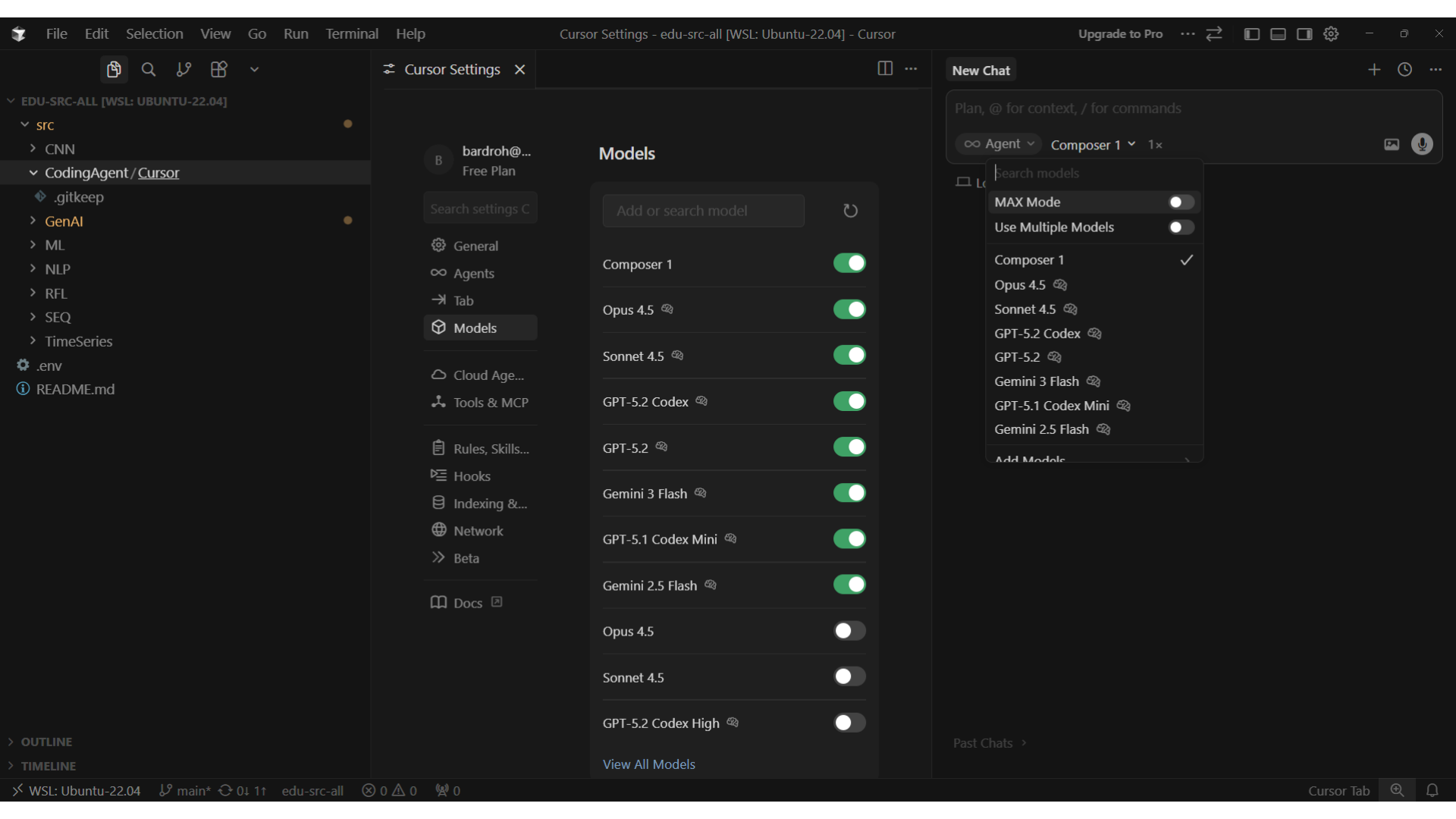
(Press Esc to close)

- Gemini CLI를 대체하는 구글의 새로운 AI 코딩 에이전트
- 리눅스라면 다음 명령으로 간단히 설치 가능

```
curl -fsSL https://antigravity.google/cli/install.sh | bash
```

- 실행은 'agy'로 하며, Gemini CLI, Claude Code 등과 유사한 인터페이스
- 최초 실행 시 로그인 요구 (Google OAuth 또는 Google Cloud 프로젝트 선택)
- Gemini CLI는 하루 1,000 요청이었으나, Antigravity CLI는 주 단위 컴퓨트 기반 상한으로 변경
- Antigravity 2.0(GUI)와 사용량(quota)을 공유: Gemini Code Assist와 Gemini CLI의 사용량을 합산한 통합 기준으로 quota가 적용

- Cursor는 Anysphere에서 개발한 AI 코드 에디터로, 프로그래밍 코드의 이해 및 생성에 특화된 혁신적인 기능을 제공
- VS Code 기반 인터페이스를 통해 개발자와 상호작용하며, 코드 작성, 버그 수정, 전체 프로젝트 탐색 등 다양한 개발 관련 작업 수행 가능
- 가입은 <https://cursor.com> 에서 가능하며, 무료(Hobby) 계정은 기본 모델을 사용하고, 한 달에 \$20의 비용(Pro)을 지불하면 고성능 모델 또는 높은 한도를 제공
- Hobby 계정은 탭 자동완성과 Agent 요청을 제한된 한도 내에서 제공, Agent 작업은 50회로 알려져 있음
- Pro(\$20), Pro+(\$60), Ultra(\$200)로 요금제 비용이 올라갈 때마다 사용량이 늘어나는 구조
- 사용자 본인의 OpenAI 또는 Anthropic API 키를 직접 입력하여 연동할 수도 있으며, 이 경우 사용한 토큰 양에 비례하여 해당 서비스사에 요금이 과금됨
- 바이브 코딩을 대중화시킨 혁신적인 개발도구라는 평가를 받지만, 최근 Claude Code, Antigravity, Gemini CLI 등 다양한 경쟁자들이 나오면서 예전의 독점적 지위는 유지하지 못하는 상황



- OpenAI가 2025년 재출시한 클라우드 기반 AI 소프트웨어 엔지니어링 에이전트로, ChatGPT 웹·Codex CLI(터미널)·IDE 확장 세 가지 경로로 접근 가능
- 별도 구독 플랜 없이 기존 ChatGPT 플랜에 포함: 헤비유저는 Plus(\$20/월), Pro(\$200/월) 또는 Business(\$30/유저/월) 권장하나, 무료 플랜도 어느 정도 사용량을 제공
- 주간 단위 quota이며, codex cli의 /status 명령이나 <https://chatgpt.com/codex/settings/usage> 에서 확인 가능
- 각 태스크가 격리된 클라우드 샌드박스에서 독립 실행되어 여러 작업을 병렬 처리 가능
- API 키 연동 시 종량제 사용 가능 (codex-mini-latest 기준 입력 \$1.5/1M 토큰, 출력 \$6/1M 토큰)
- Claude Code 대비 사용량이 많고, 특히 Pro 요금제는 거의 무제한에 가까운 요청을 수행할 수 있다는 평가
- gpt-5.5 출시 이후 재평가 되고 있는 AI 코딩 에이전트로, Claude Code의 경쟁자로 인식되고 있음

Extension: Codex – OpenAI's coding agent X



Codex – OpenAI's coding agent

OpenAI openai.com | 4,347,869 | (260)

Codex is a coding agent that works with you everywhere you code — included in ChatGPT Plus, Pro, Business, Edu, and Enterprise plans.

Disable ▾ Uninstall ▾ **Switch to Pre-Release Version** ✓ Auto Update ⚙

Extension is enabled on "WSL: Ubuntu-22.04"

DETAILS FEATURES

Codex is OpenAI's coding agent that helps you write, review, and ship code faster. Use it side-by-side in your IDE or delegate larger tasks to the cloud.

Pair with Codex

Add Codex as a panel in VS Code to chat, edit, and preview changes seamlessly. With context from opened files and selected code, you can write shorter prompts and get faster, more relevant results. (Tip: many users like moving the Codex panel to the right side of the editor.)

Delegate to Codex in the cloud

Offload bigger jobs to Codex in the cloud, then track progress and review results without leaving your IDE. For finishing touches, open cloud tasks locally. Codex keeps the context consistent.

Sign in with your ChatGPT account

Codex works with Plus, Pro, Business, Edu, and Enterprise ChatGPT plans. Learn what's included in your plan at the [Help Center](#).

Learn more about Codex

See the docs at <https://developers.openai.com/codex/ide>.

(MacOS only) Work with the ChatGPT macOS app

The OpenAI Codex extension also supports connecting the ChatGPT macOS app to VS Code so ChatGPT can answer questions or make simple edits. To use this feature, launch the ChatGPT macOS and select "Work with VS Code".

Marketplace

Identifier	openai.chatgpt
Version	0.4.74
Published	11 months ago
Last Released	13 hours ago

Categories

AI Chat

Resources

- License
- OpenAI
- Marketplace

CA tetris

Tip: GPT-5.5 is now available in Codex. It's our strongest agentic coding model yet, built to reason through large codebases, check assumptions with tools, and keep going until the work is done.

Learn more: <https://openai.com/index/introducing-gpt-5-5/>

/status

>_ OpenAI Codex (v0.129.0)

Visit <https://chatgpt.com/codex/settings/usage> for up-to-date information on rate limits and credits

Model: gpt-5.5 (reasoning medium, summaries auto)

Directory: ~/work/tetris

Permissions: Workspace (on-request)

Agents.md: <none>

Account: bardroh@gmail.com (Free)

Collaboration mode: Default

Session: 019e05a8-2cd6-7a11-87f0-1c448f167485

Weekly limit: [] 95% left (resets 10:52 on 15 May)

> Find and fix a bug in @filename

gpt-5.5 default · ~/work/tetris

GitHub 계정이 연결되어 있지 않습니다. [지금 연결하세요.](#)

설정

일반

환경

코드 검토

커넥터

분석

데이터 제어

Codex 애널리틱스

7일

1개월

사용자 지정

그룹화 기준: **일별**

사용 현황 코드 리뷰

잔고

Codex 사용량은 공유 에이전틱 사용 한도에서 차감됩니다 ①

주간 사용 한도

95% 남음



2026. 5. 15. 오전 10:50 초기화

사용량 세부 정보 ①

개인 사용량

이 기간에 데이터가 없습니다

```
sudo apt update && sudo apt upgrade -y
```

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash # nvm 설치
```

```
export NVM_DIR="$HOME/.nvm"
```

```
source "$NVM_DIR/nvm.sh"
```

```
nvm install --lts # node 설치
```

```
node -v # v24.x 확인
```

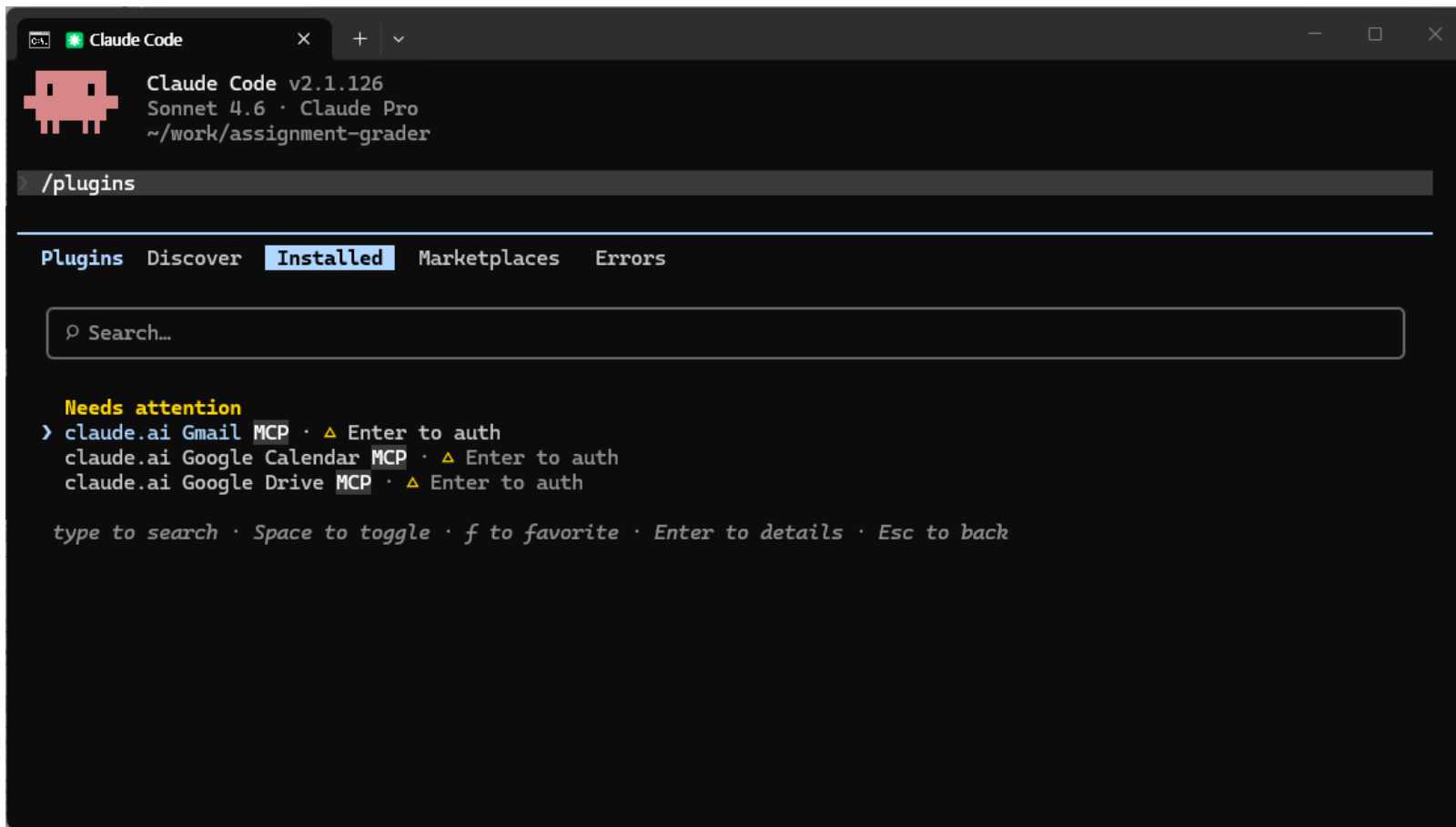
```
npm -v
```

```
npm install -g @openai/codex # codex 설치
```

```
cd your-project
```

```
codex # codex login
```

- Anthropic이 개발한 터미널(CLI) 기반 AI 코딩 에이전트로, IDE가 아닌 명령줄에서 동작하며 코드베이스 전체를 읽고 파일 편집·테스트 실행·Git 커밋까지 자율적으로 수행
- 가입은 claude.ai에서 가능하며, Pro(\$20/월)부터 Claude Code 이용 가능, 고사용자는 Max 5×(\$100/월) 또는 Max 20×(\$200/월) 선택
- 무료 플랜은 Claude Code 미지원, Pro 플랜은 웹·앱·터미널 한도를 공유하므로 헤비 유저는 빠르게 소진됨
- 자신의 Anthropic API 키를 직접 입력해 토큰 사용량에 비례한 종량제로도 사용 가능
- CLAUDE.md, Skill, Command, Agent 도입 등 AI 코딩 에이전트의 사실상 표준: Claude Code가 도입하면 다른 에이전트들이 지원하는 양상
- 기본 사용량 축소와 잦은 장애로 사용자들의 불만이 누적: 그러나 그만큼 많이 사용되는 에이전트라는 증거
- 신뢰할 수 있는 수준의 결과물을 내는 몇 개 안되는 AI 코딩 에이전트라는 평가: 사용이 빠르게 확산되는 추세





CLAUDE CODE

+ New session

Local

Web

🔍 Search sessions...

- /effort 1d
- Design custom grader container strategy 4d
- Include agent execution logs in raw JSON 4d
- Assess agent mode implementation feasibility   5d
- LLM-based autonomous evaluation agent design 5d
- Adjust terms dialog indent for emphasis 5d
- GitHub 로그인 시 약관 동의 플로우 구성 5d
- Add custom Docker image support with credent... 5d
- Review mode LLM multi-rubric handling 5d
- Git repository grading workflow discussion 5d
- Rename requirements folder to discussion 5d
- 방지 방법 검토 5d
- Design AI-powered instructing system to replac... 5d
- Advanced evaluation process with in-depth cod... 7d
- Analyze why tech companies outsource internal ... 7d
- Build landing page with contact inquiry flow 8d
- Prepare education institution meeting presentat... 8d
- Evaluate Claude Code agent workflow improve... 8d
- Evaluate Claude coding system limitations 10d
- Fix font inconsistency in scoring screen 10d
- Plan instructor override feature for item-level gr... 10d

Claude Code ✖

Claude Code



Press `Shift` `Tab` to automatically approve code edits

 Prefer the Terminal experience? Switch back in Settings. ✖

ctrl esc to focus or unfocus Claude



</> Edit automatically



```
sudo apt update && sudo apt upgrade -y
```

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash # nvm 설치
```

```
export NVM_DIR="$HOME/.nvm"
```

```
source "$NVM_DIR/nvm.sh"
```

```
nvm install --lts # node 설치
```

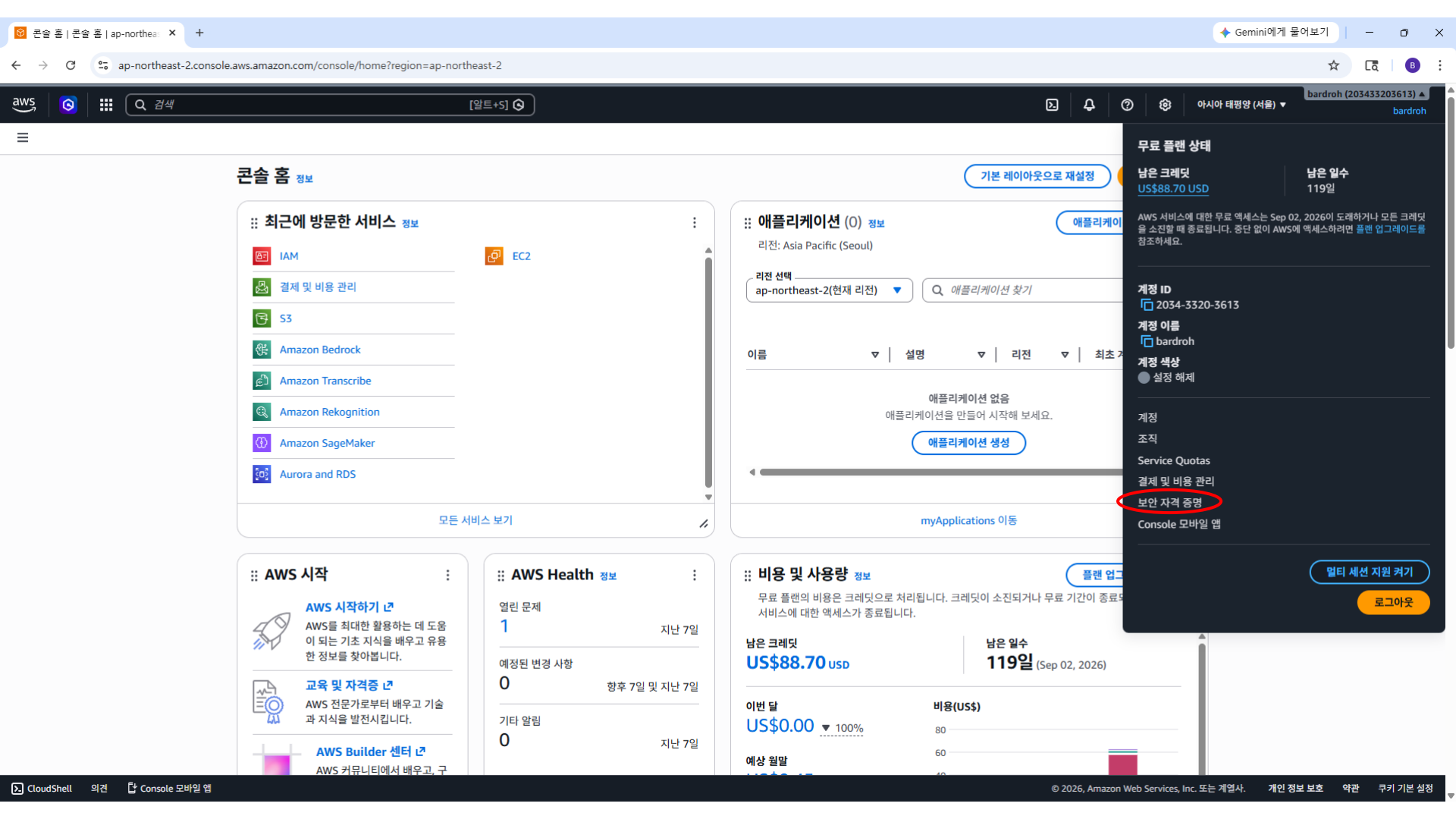
```
node -v # v24.x 확인
```

```
npm -v
```

```
npm install -g @anthropic-ai/claude-code # claude code 설치
```

```
cd your-project
```

```
claude
```

보안 자격 증명 | IAM | Global

Gemini에게 물어보기

us-east-1.console.aws.amazon.com/iam/home?region=ap-northeast-2#/security_credentials

aws

검색

[알트+S]

bardroh (203433203613)

bardroh

IAM > 보안 자격 증명

Identity and Access Management(IAM)

IAM 검색

대시보드

▼ 액세스 관리

역할

정책

IAM 사용자

IAM 사용자 그룹

ID 제공업체

계정 설정

루트 액세스 관리

임시 위임 요청

▼ 보고서 액세스

Access Analyzer

리소스 분석

미사용 액세스

분석기 설정

자격 증명 보고서

조직 활동

서비스 제어 정책

리소스 제어 정책

관련 콘솔

IAM 자격 증명 센터

AWS Organizations

내 보안 자격 증명

루트 사용자 정보

루트 사용자는 이 계정의 모든 AWS 리소스에 액세스할 수 있으며 다음 모범 사례를 수행하는 것이 좋습니다. AWS 자격 증명의 유형과 사용 방식에 대해 자세히 알아보려면 AWS 일반 참조의 AWS 보안 자격 증명을 참조하세요.

계정 세부 정보

작업

계정 이름

bardroh

AWS 계정 ID

203433203613

이메일 주소

sales@weable.ai

정규 사용자 ID

622c0b4fcf66f40ec9812c9344041e63d7aea7e1f63fb62a2b1365bd9d5bf8f9

멀티 팩터 인증(MFA) (1)

삭제

재동기화

MFA 디바이스 할당

MFA를 사용하여 AWS 환경의 보안을 강화합니다. MFA로 로그인하려면 MFA 디바이스의 인증 코드가 필요합니다. 각 사용자는 MFA 디바이스를 최대 8개까지 할당할 수 있습니다. 자세히 알아보기

유형	식별자	인증	생성 날짜
☐ 가상	arn:aws:iam:203433203613:mfa/Authapp	해당 사항 없음	Tue Apr 07 2026

엑세스 키 (1)

작업

엑세스 키 만들기

엑세스 키를 사용하여 AWS CLI, AWS Tools for PowerShell, AWS SDK 또는 직접 AWS API 호출을 통해 AWS에 프로그래밍 방식 호출을 전송합니다. 한 번에 최대 두 개의 액세스 키(활성 또는 비활성)를 가질 수 있습니다. 자세히 알아보기

엑세스 키 ID	생성 날짜	마지막으로 사용한 액세스 키	마지막으로 사용된 리전	마지막으로 사용한 서비스	상태
☐ AKIA56XMQK6OU4DWORHO	54일 전	11일 전	us-east-1	bedrock	Active

CloudFront 키 페어 (0)

작업

업로드

CloudFront 키 페어 생성

Amazon CloudFront의 키 페어를 사용하여 서명된 URL을 생성합니다. 한 번에 최대 두 개의 CloudFront 키 페어(활성 또는 비활성)를 보유할 수 있습니다.

생성 시간	CloudFront 키 ID	상태
-------	-----------------	----

CloudFront 키 페어 없음

© 2026, Amazon Web Services, Inc. 또는 계열사.

개인 정보 보호

약관

쿠기 기본 설정

- GitHub에서 제공하는 정적 웹사이트 호스팅 서비스
- 별도의 서버 없이 HTML, CSS, JavaScript로 바로 배포 가능
- Git 저장소의 내용을 그대로 웹사이트로 제공
- username.github.io 형태의 기본 도메인 제공
- 브랜치(main, gh-pages) 또는 특정 폴더(/docs) 기반 배포
- 커밋과 동시에 자동 배포되는 간단한 워크플로우
- 개인 랜딩 페이지, 포트폴리오, 프로젝트 소개에 적합
- 백엔드 로직 없이 프론트엔드 결과물 공유에 최적
- 바이브 코딩 결과물을 가장 빠르게 공개할 수 있는 방법

- Github에 로그인하고 배포용 public repo 생성 (private repo는 업그레이드 필요)
- 다음 명령들로 로컬 repo의 소스를 원격에 push

```
git init
```

```
git add .
```

```
git commit -m "Initial commit: Tetris game"
```

```
git remote add origin git@github.com:  
[계정명]/[repo이름].git
```

```
git branch -M main
```

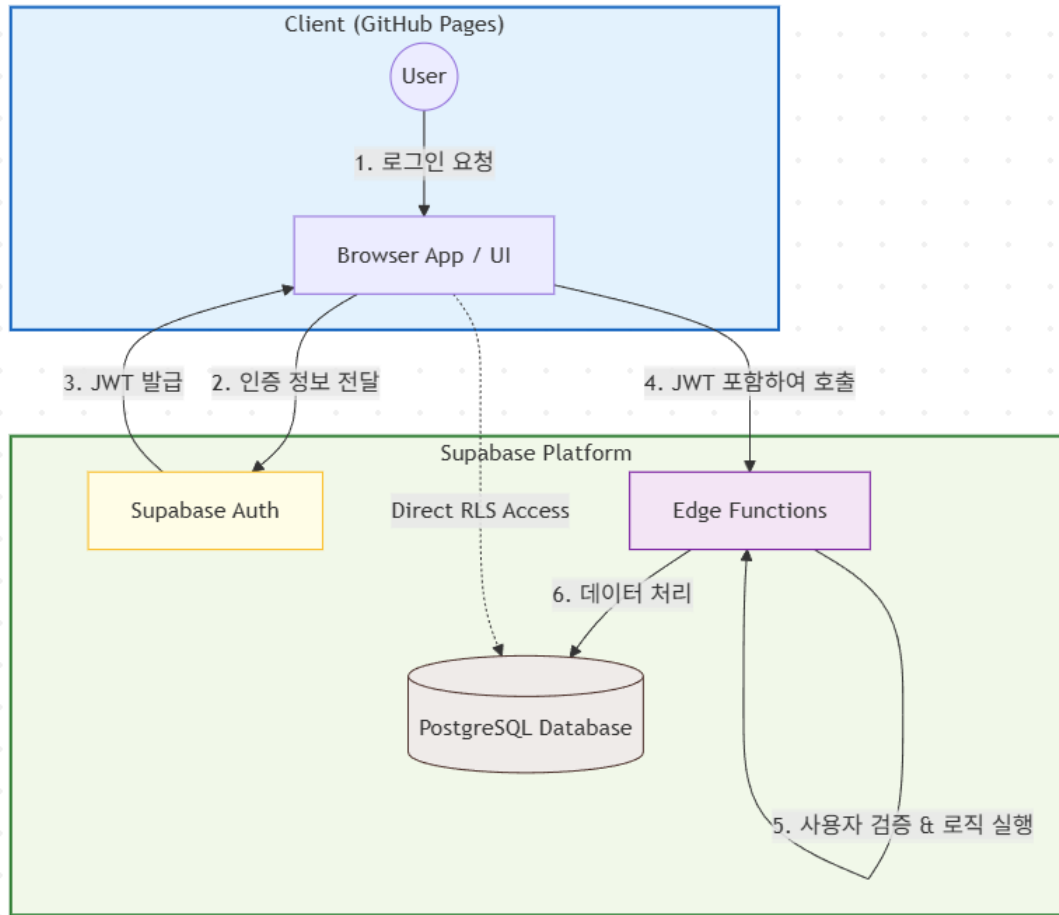
```
git push -u origin main
```

- 배포용 repo의 Settings-Pages로 이동
- Source 섹션에서 branch를 main으로, 폴더를 "/"(root)으로 선택하고 Save
- 1~2분 정도 시간이 걸린 후 Pages 설정에서 사이트 주소 확인 가능
- 기본 주소는 [https://\[계정명\].github.io/\[repo명\]](https://[계정명].github.io/[repo명])
- 도메인을 보유하고 있다면, Custom domain에서 연결할 수 있음

- Firebase의 오픈소스 대안을 지향하는 Backend-as-a-Service
- PostgreSQL 기반의 관계형 데이터베이스 제공
- 인증(Auth), DB, 스토리지, 실시간 기능을 통합 제공
- SQL을 그대로 사용할 수 있어 백엔드 확장성이 높음
- Row Level Security(RLS)로 세밀한 접근 제어 가능
- 프론트엔드에서 SDK로 직접 데이터 접근 가능
- GitHub Pages 같은 정적 사이트와 궁합이 좋음
- 개인 프로젝트부터 스타트업 MVP까지 활용 가능
- 서버를 직접 운영하지 않고도 풀스택 개발 가능
- 바이브 코딩으로 만든 앱에 백엔드 생명력을 부여

- Supabase에서 제공하는 서버리스 함수 실행 환경
- Deno 기반으로 TypeScript/JavaScript 작성
- 전 세계 Edge에서 실행되어 낮은 지연시간 제공
- REST API 형태로 프론트엔드에서 호출 가능
- DB 트리거, 외부 API 연동, 비즈니스 로직 처리에 사용
- 인증 토큰과 RLS를 함께 활용 가능
- “백엔드 서버 하나 띄우기” 부담 없이 로직 구현
- GitHub Pages + Supabase의 백엔드 연결 고리 역할

- Supabase는 JWT(Json Web Token) 기반 인증을 사용
- 인증은 Supabase Auth(GoTrue)가 전담하여 처리
- 로그인 성공 시 access token(JWT) 과 refresh token 발급
- API 요청 시 JWT를 Authorization: Bearer 헤더로 전달
- 프론트엔드에는 anon public key + 사용자 JWT만 사용
- JWT에는 사용자 ID, 역할(role), 만료 시간 정보 포함
- 로그인하지 않은 경우 JWT는 존재하지 않음
- 데이터 접근 제어는 Row Level Security(RLS) 로 수행
- JWT는 “권한”이 아니라 사용자 식별 수단
- 보안의 핵심은 JWT + RLS 정책의 결합



- Vercel이 개발한 프론트엔드 특화 클라우드 배포 플랫폼
- Next.js, React, Vue, Svelte 등 주요 프레임워크 공식 지원
- 별도 서버 설정 없이 코드 푸시만으로 즉시 배포 가능
- 글로벌 CDN 기반으로 전 세계 어디서나 빠른 응답 속도 제공
- 무료 플랜으로도 개인 프로젝트 및 포트폴리오 운영 가능
- vercel.app 형태의 기본 도메인 자동 제공, 커스텀 도메인 연결 지원
- 프론트엔드 개발자가 인프라 걱정 없이 서비스에 집중할 수 있는 환경

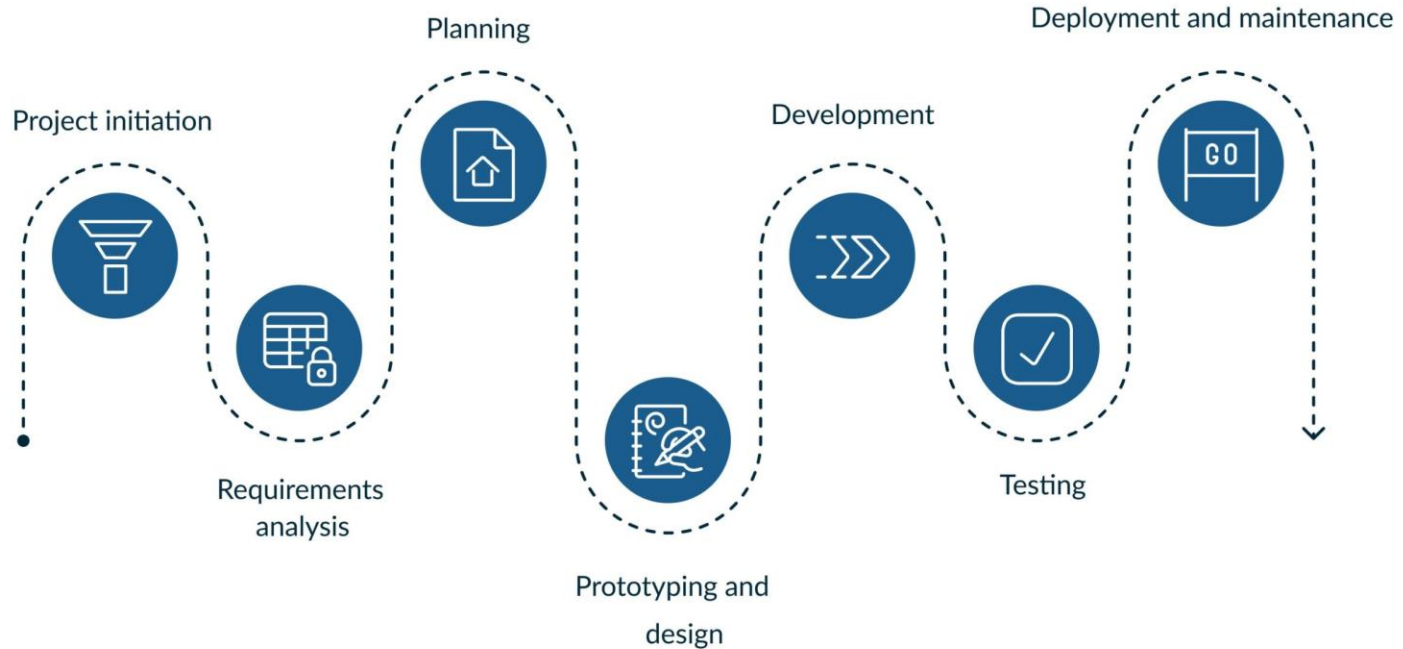
- GitHub / GitLab / Bitbucket 저장소와 연동하여 배포 시작
- 저장소 import 후 프레임워크 자동 감지 및 빌드 설정 자동 완성
- main 브랜치에 푸시하면 자동으로 프로덕션 배포 트리거
- PR(풀 리퀘스트) 생성 시 Preview URL 자동 생성 — 배포 전 결과 확인 가능
- 빌드 로그 실시간 확인, 실패 시 즉각적인 오류 메시지 제공
- 배포 히스토리 관리 및 이전 버전으로 즉시 롤백 가능
- 바이브 코딩 결과물을 git push 한 번으로 즉시 공개 가능

- Serverless Function 지원 - 간단한 백엔드 API를 별도 서버 없이 구현 가능
- 환경 변수(Environment Variables) 설정으로 API 키 등 민감 정보 안전하게 관리
- Edge Network 기반 배포로 사용자 위치에 가장 가까운 서버에서 응답
- Analytics 기능으로 트래픽, 성능 지표를 대시보드에서 한눈에 확인
- 팀 협업 기능 지원 — 조직 단위 프로젝트 관리 및 권한 설정 가능
- Next.js와의 완벽한 통합 — SSR, ISR, Image Optimization 등 풀 지원
- CLI 도구 제공으로 터미널에서도 간편하게 배포 및 관리 가능

- npm을 통해 Vercel CLI 전역 설치: `npm install -g vercel`
- 터미널에서 `vercel login` 명령어로 Vercel 계정 인증 (브라우저 자동 실행)
- 배포할 프로젝트 폴더로 이동 후 `vercel` 명령어 입력으로 배포 시작
- 최초 실행 시 프로젝트 이름, 프레임워크, 빌드 설정 등을 대화형으로 구성
- 설정 완료 후 자동 빌드 및 Preview URL 생성(인증 필요) - 배포 결과 즉시 확인 가능
- `vercel --prod` 명령어로 프로덕션 환경에 직접 배포
- 두 번째 배포부터는 `vercel` 또는 `vercel --prod` 한 줄로 즉시 재배포
- `vercel env add` 로 환경 변수 CLI에서 직접 추가 및 관리 가능
- git 연동 없이도 로컬 파일을 바로 배포할 수 있어 빠른 테스트에 적합

- Claude Code에서 AI의 행동 규칙을 정의하는 설정 파일
- CLAUDE.md에 정의할 수 있음
- 적용 범위는 파일 위치로 결정됨: 루트의 ~/CLAUDE.md는 모든 프로젝트에 전역 적용, 프로젝트 루트의 ./CLAUDE.md는 해당 프로젝트 한정, 하위 디렉토리의 ./src/CLAUDE.md처럼 폴더 별로도 설정 가능 (해당 디렉토리 작업 시만 적용)
- AI가 코딩 스타일, 기술 스택, 제약 조건을 일관되게 따르도록 유도
- "항상 TypeScript 사용", "테스트 코드 우선 작성" 같은 규칙 지정 가능
- 팀 단위로 AI 사용 방식 표준화에 효과적 (CLAUDE.md를 git으로 팀 공유)
- 프롬프트를 매번 입력하지 않아도, Claude Code 세션 시작 시 자동으로 컨텍스트 반영
- 바이브코딩 시 코드 품질 흔들림을 줄여줌
- 사람의 코드 컨벤션 + AI의 생산성을 결합

7 Stages of Application Development Life Cycle



- CLAUDE.md에서 참조하는 절차적 지식 파일로, AI가 특정 도메인/작업을 수행하는 방법을 정의
- 각 Skill은 SKILL.md 파일을 포함하는 폴더 단위로 구성되며, 지침·스크립트·명령·템플릿 등을 포함
- CLAUDE.md에 Skill 경로를 명시해두면, Claude Code가 관련 작업 시 해당 SKILL.md를 읽어 절차를 따름 - 즉 필요할 때만 동적으로 로딩되어 컨텍스트를 효율적으로 사용
- CLAUDE.md가 "무엇을 지켜야 하는가"의 정적 제약이라면, Skill은 "어떻게 수행할 것인가"의 절차적 How-to 지식: ~할 때는 ~한다
- Claude Code 세션에서 @경로/SKILL.md로 직접 참조하거나, CLAUDE.md에 조건부 로딩 규칙을 기술하는 방식으로 활용
- 스크립트·템플릿·명령을 포함해 데이터베이스 마이그레이션, API 생성, 테스트 워크플로우 같은 복잡한 자동화도 캡슐화 가능
- 필요할 때만 활성화되므로 토큰/컨텍스트 비용 절약에도 효과적
- 도메인 전문 지식을 팀 자산으로 축적하고, git으로 공유해 AI 작업 방식을 조직 수준에서 표준화 가능

데이터베이스 마이그레이션 스킬

마이그레이션 파일 생성, 스키마 변경, 데이터 이관 작업 요청 시 이 가이드를 따릅니다.

파일 명명 규칙

마이그레이션 파일은 `YYYYMMDD\_HHMMSS\_description.sql` 형식으로 작성합니다.

필수 사항

1. 모든 마이그레이션은 롤백(down) 스크립트를 함께 작성합니다
2. 대량 데이터 변경 시 배치 처리를 고려합니다
3. 인덱스 추가는 CONCURRENTLY 옵션을 사용합니다

예제

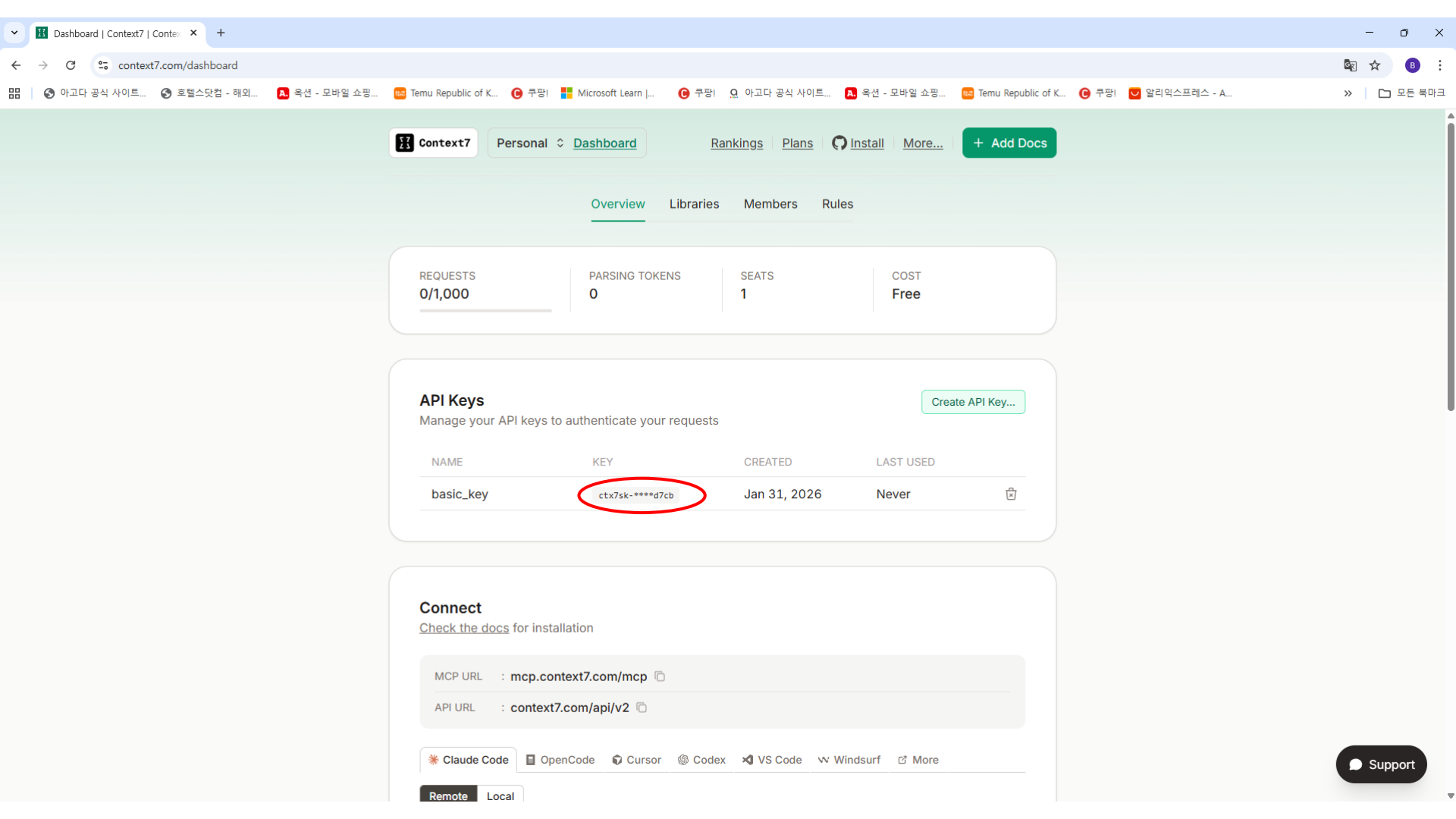
examples/ 디렉토리의 예제를 참고하세요.

- Claude Code에서 메인 에이전트가 큰 작업을 작은 하위 작업으로 나눠 처리하도록 돕는 독립적 에이전트
- Claude Code는 Task 툴을 통해 서브에이전트를 실행하며, 각 서브에이전트는 별도의 컨텍스트 윈도우를 가지고 독립적으로 작동
- 각 서브에이전트는 전용 프롬프트와 툴 액세스를 가질 수 있으며, 메인 에이전트와 격리된 환경에서 실행됨
- Claude Code는 기본적으로 파일 탐색, 터미널 실행, 웹 검색 등의 작업을 서브에이전트로 병렬 처리 가능
- 복잡한 작업에서 세부 전문화된 처리가 가능해짐 - 예) 테스트 실행·lint·build를 동시에 병렬 처리
- 결과적으로 메인 에이전트는 간결한 컨텍스트 유지 + 전문 작업 분리가 이루어짐
- 특정 하위 작업이 병렬로 처리되면 전체 작업 시간이 단축될 수 있음
- CLAUDE.md에 서브에이전트 활용 방식을 명시하거나, `claude --dangerously-skip-permissions` 플래그와 조합해 자율 실행 파이프라인 구성도 가능
- 큰 코드베이스나 복잡한 플랜에서 실행/전문화 효율을 크게 높이는 구조적 개선이며, Claude Code가 Agentic 루프를 도는 핵심 메커니즘 중 하나

- Claude Code에서 md 파일로 정의할 수 있는 커스텀 슬래시 명령
- .claude/commands/ 폴더에 마크다운 파일을 저장하면 Claude Code 세션에서 /명령어명으로 즉시 실행 가능. 전역으로 쓰려면 ~/.claude/commands/에 저장
- 명령 파일 안에 \$ARGUMENTS 변수를 사용하면 실행 시 인자를 동적으로 전달 가능
- 예) /deploy production → 파일 내 \$ARGUMENTS가 production으로 치환
- CLAUDE.md가 "항상 지켜야 할 제약"이고, Skill이 "어떻게 할지의 절차 지식"이라면, Command는 "지금 당장 실행"하는 트리거 역할
- git으로 .claude/commands/를 팀과 공유하면 팀 전체의 반복 작업을 표준화할 수 있고, 온보딩 비용도 줄일 수 있음
- Commands는 내부적으로 CLAUDE.md나 Skill의 내용을 참조하도록 구성할 수 있어, 세 가지를 조합하면 규칙 -> 절차 -> 실행의 완성된 자동화 레이어가 만들어짐

구분	Command	Skill	SubAgent
핵심 개념	즉시 실행하는 단발 명령	절차적 How-to 지식 문서	특정 역할을 가진 독립 실행 단위
실행 방식	/명령어명	@SKILL.md 참조 또는 자동 적용	Task 툴 호출
지속성 (Memory)	없음	없음 (절차 문서)	있음 (역할·목표 단위)
적용 범위	현재 세션	프로젝트 전반	작업 단위 또는 워크플로우
자동 개입	X	O	O
역할 정체성	없음	중간 (절차·How-to)	강함 (격리된 실행 단위)
컨텍스트 적용	현재 세션	누적 컨텍스트	독립 컨텍스트 윈도우
병렬 작업	X	X	O
실패 격리	X	△	O
복잡도	낮음	중간	높음
저장 경로	.claude/commands/	SKILL.md 폴더	Task 툴 내 인라인 정의
주요 용도	설명, 수정, 요약	개발 규칙, 반복 패턴	설계·검토·대규모 작업

- MCP는 LLM이 도구를 연동하는 표준적인 방법으로, Claude Code에 이러한 MCP 서버들을 연동하면 더욱 기능이 풍부해지는 효과가 있음
- Claude Code에서 사용할 수 있는 MCP 서버 중 대표적인 것이 Context7
- <https://context7.com/> 에서 계정을 만들고 key를 받아서 사용할 수 있음
- Context7가 하는 일은, 개발에 필요한 라이브러리의 최신 버전별 문서와 코드 예제를 소스 코드에서 직접 가져와 프롬프트에 주입하는 것
- 이를 통해 코딩작업에 오래된 라이브러리를 사용하면서 발생하는 문제들의 상당부분이 해소
- Rules에서 항상 Context7을 사용하게 할 수도 있고, "Context7 MCP를 사용해줘" 처럼 프롬프트에서 명시적으로 사용을 요청할 수도 있음
- 가져온 라이브러리 문서와 예제가 프롬프트에 포함되기 때문에, 토큰 사용량을 좀더 늘어날 수 있음



- Spec-Driven Development: 전통적 코드 중심 개발에서 사양 기반 개발로 전환하자는 개발방법론
- 바이브코딩이 대중화되면서, Spec(사양서)이 스펙이 곧 실행 가능 문서로서 구현을 자동 생성하거나 안내하는 중요한 역할이라는 점이 급부상
- Speckit은 이런 SDD를 가능하게 해주는 도구로서, 다양한 AI 코딩 agent들을 지원
- 'specify init <repo_name>'으로 초기화할 수 있으며, Claude Code를 선택하면 .claude/commands/ 폴더 밑에 Command로 저장됨
- speckit.analyze.md, speckit.clarify.md, speckit.implement.md, speckit.specify.md, speckit.taskstoissues.md, speckit.checklist.md, speckit.constitution.md, speckit.plan.md, speckit.tasks.md의 9개 명령
- 초기화 후 /speckit.constitution(프로젝트 헌법 제정), /speckit.specify(사양 정의), /speckit.plan(계획 수립), /speckit.tasks(작업을 Task로 분할), /speckit.implement(구현)의 순서로 작업 진행

- Command, SubAgent, MCP 서버, Hook 등을 하나의 패키지로 묶어서 팀·프로젝트 간에 쉽게 공유할 수 있는 확장 시스템
- Claude Code 내에서 /plugin 명령어로 직접 설치 가능 (현재 퍼블릭 베타)
- 설치 방법: /plugin install {플러그인명}@{마켓플레이스명} 또는 /plugin > Discover에서 탐색
- 플러그인 구성 요소: Skills(커스텀 커맨드/프롬프트), MCP 서버(외부 API 연결), Hooks(이벤트 기반 자동 실행 스크립트)
- 표준 구조: .claude-plugin/plugin.json(메타데이터), commands/, agents/, skills/, hooks/, .mcp.json 으로 구성
- 코드 리뷰 표준 강제, 배포 파이프라인 공유, 내부 도구 MCP 연결, 프레임워크별 커스터마이징 번들링 가능
- 공식 마켓플레이스 외에 커뮤니티 마켓플레이스도 존재 (npx claude-plugins install로도 설치 가능)

- Geoffrey Huntley가 고안한 "Ralph Wiggum 기법"에서 유래: Claude가 완료 조건을 충족할 때까지 동일한 프롬프트를 반복 실행하는 자율 루프 패턴: 앞서 설명한 plugin으로 배포
- 세션 종료 시도를 Stop Hook이 가로채어, 이전 파일 수정과 git 히스토리를 유지한 채 프롬프트를 자동으로 재주입하는 방식으로 동작
- 실행 흐름: /ralph-loop "작업설명" 입력 -> Claude Code 작업 수행 -> 종료 시도 -> Stop Hook이 차단 -> 동일 프롬프트 재주입 -> 완료 조건 충족 시 종료
- 기본 사용법: --completion-promise 옵션으로 완료 조건 텍스트 지정, --max-iterations로 최대 반복 횟수 제한
- 테스트 실패, 이전 시도 결과를 바탕으로 Claude Code가 스스로 작업을 개선하는 자율적 피드백 루프 형성
- 명확한 성공 기준이 있는 작업, 자동화된 검증이 가능한 Green Field 개발, 반복 정제가 유효한 태스크에 적합
- 단일 컨텍스트 창에서 계속 실행되므로, 반복이 많아질수록 컨텍스트가 팽창하고 환각 위험이 증가할 수 있음

- Harness Engineering이란, AI 에이전트가 안정적으로 작업할 수 있도록 실행 환경과 검증 체계를 설계하는 엔지니어링 영역을 의미
- LLM에 기반한 작업은 Prompt Engineering -> Context Engineering -> Harness Engineering으로 발전해왔음
- Prompt Engineering: 프롬프트를 어떻게 구성하느냐가 LLM의 응답 품질을 결정한다. CoT, Few Shot 등의 기법을 활용하여 어떻게 의도에 맞는 프롬프트를 구성하느냐가 목표
- Context Engineering: 주어진 Context에 작업에 필요한 자료들을 어떻게 효율적으로 채워넣느냐를 중요하게 봄. Context가 너무 길면 Lost-in-the-Middle 문제가 발생하고, 비용과 속도면에서 효율적이지 않음
- Harness Engineering: 여러 번 API 호출을 반복하면서 자율적으로 구동하는 AI agent를 위한 환경을 어떻게 구축하느냐(작업 정의, 환경 구성, 검증 체계, 실행 기록, 재시작 루프 등)가 결과물의 품질을 결정. 대규모의 코드베이스를 직접 다루는 작업에서 중요시됨

- Task Definition(작업 정의): 어떤 작업을 할 것인가를 모호한 자연어가 아닌 확실하게 정의된 문서로 제공. PRD, TRD, Tasks, Domain Language 등을 정의
- Runtime Environment (실행 환경): docker runtime 등 독립된 SandBox 환경을 제공하면, 외부 환경변화에 관계없이 반복 실행 시에도 동일한 결과를 도출할 수 있음
- Test & Eval Harness (검증 체계): 사실 상 AI agent에서 구동 가장 중요한 요소로, 어떻게 비결정적인 LLM의 출력을 결정적으로 바꾸느냐를 판단하는 요소. UnitTest(TDD), Playwright, lint 등 다양한 요소로 작업의 성공 여부를 판단하게 함
- Logging / Trace (실행 기록): 실행 기록을 로그로 남기면, 이후 작업 재개나 실패시의 복구시 원인 분석을 위한 중요한 단초가 됨
- Retry & Feedback Loop (재시도 및 수정 루프): 추론-실행-검증을 반복하면서, 원인이 무엇인지를 파악하고 수정하여 목표를 달성하는 닫힌 루프를 구성. 앞서 설명한 /ralph-loop가 대표적
- Claude Code 자체가 잘 구성된 Harness의 대표적 사례로, 위에서 언급한 모든 구성요소들을 충실하게 구현

mattpocock (Matt Pocock)

github.com/mattpocock

Google에 물어보기

mattpocock

Type [] to search

Overview

Repositories220

Projects

Packages

Stars295



Matt Pocock

mattpocock

Follow

Sponsor

TypeScript wizard. Building Total TypeScript - a comprehensive, production-grade TypeScript course. Ex-Vercel, Stately.

14.1k followers · 33 following

Total TypeScript

Oxfordshire

https://totaltypescript.com

@mattpocockuk

Sponsors



mattpocock / readme.md



Pinned

skillsPublic

Skills for Real Engineers. Straight from my .claude directory.

Shell63.3k5.5k

sandcastlePublic

Orchestrate sandboxed coding agents in TypeScript with sandcastle.run()

TypeScript3.8k356

dictionary-of-ai-codingPublic

AI coding jargon, explained in plain English.

TypeScript1.2k140

ts-resetPublic

A 'CSS reset' for TypeScript, improving types for common JavaScript API's

TypeScript8.5k132

- Matt Pocock이 실제 .claude 디렉토리에서 사용하는 에이전트 Skills 모음 (현재 GitHub Star 62.6k)
- GSD, BMAD 같은 "프로세스 주도형" 접근 대신, 작고 조합 가능한 단위 Skills를 지향
- 문제 1 - 의견 불일치: 사용자가 원하는 걸 에이전트가 만들지 못하는 가장 큰 원인. /grill-me, /grill-with-docs로 작업 시작 전 심층 질의(grilling) 세션 수행
- 문제 2 - 언어 불일치: 프로젝트 전용 용어(Domain Language)가 없으면 에이전트가 사용자의 요청을 혼동. CONTEXT.md로 공유 언어(Ubiquitous Language) 구축, 변수명·함수명까지 일관성 확보
- 문제 3 - 동작하지 않는 코드: 피드백 루프(검증) 부재가 원인. /tdd로 red-green-refactor 사이클 강제, /diagnose로 체계적 디버깅
- 문제 4 - 진흙 덩어리(Big Ball of Mud): 에이전트가 코딩 속도를 높이면 소프트웨어 엔트로피도 가속. 즉 기술부채가 계속 쌓이게 됨. /improve-codebase-architecture를 수일마다 실행해 설계 품질 유지
- 30초 설치: `npx skills@latest add mattpocock/skills` -> 원하는 Skills 선택 -> `/setup-matt-pocock-skills` 실행

- /grill-with-docs: 계획을 기존 도메인 모델에 맞게 검증, CONTEXT.md·ADR 인라인 업데이트
- /tdd: 수직 슬라이스 단위 TDD; 기능 추가와 버그 수정 모두 적용
- /diagnose: 재현 → 최소화 → 가설 → 계측 → 수정 → 회귀 테스트 순서의 디버깅 루프
- /to-prd -> /to-issues : 대화 맥락을 PRD로 변환 후, 독립적으로 처리 가능한 GitHub 이슈로 분해
- /zoom-out: 코드 영역을 시스템 전체 관점에서 설명하도록 에이전트에 지시
- /prototype: 터미널 앱 또는 다중 UI 변형으로 throwaway 프로토타입 생성, 설계 검증용
- /caveman: 토큰 사용량을 75%까지 절감하는 초압축 커뮤니케이션 모드
- /improve-codebase-architecture: CONTEXT.md·ADR 기반으로 코드베이스 설계 개선 기회 탐색
- /git-guardrails-claude-code: 위험한 git 명령(push, reset --hard 등) 실행 전 차단 혹 설정



Home



Explore



Notifications



Follow



Chat



Grok



Bookmarks



Creator Studio



Premium 50% off



Profile



More

Post



Bard K. Roh
@bardroh



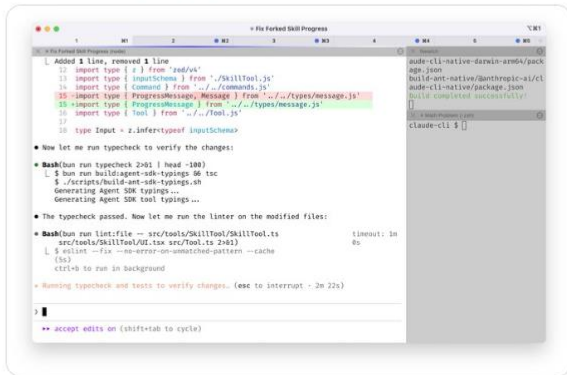
Post



Boris Cherny
@bcherny



1/ I run 5 Claudes in parallel in my terminal. I number my tabs 1-5, and use system notifications to know when a Claude needs input code.claude.com/docs/en/termin...



4:58 AM · Jan 3, 2026 · 1.1M Views



113



232



3.2K



2.4K



Relevant ▾

View quotes ▸



Post your reply



Reply



Boris Cherny
@bcherny · Jan 3



2/ I also run 5-10 Claudes on claude.ai/code, in parallel with my local Claudes. As I code in my terminal, I will often hand off local sessions to web (using &), or manually kick off sessions in Chrome, and sometimes I will -- teleport back and forth. I also start a few

[Show more](#)

Search

Relevant people



Boris Cherny
@bcherny
Claude Code @anthropicai

Follow

What's happening

올버거엔원스 X 송강

봤어? 이렇게 치킨버거야. 치킨러버의 버거, 180g
매가급 통다리살 치킨패티!
Promoted by 올버거엔원스

Trending in South Korea
신종오 판사

Trending in South Korea
#매가하우스공식스토어

Trending in South Korea
옛지러너

[Show more](#)

[Terms of Service](#) | [Privacy Policy](#) | [Cookie Policy](#) |
[Accessibility](#) | [Ads info](#) | [More...](#) © 2026 X Corp.



- Plan 모드로 시작: 계획이 마음에 들 때까지 반복 논의 후, auto-accept 모드로 전환해 1-shot 완료
- Opus 4.5 + Thinking 모드 사용: 느리지만 steering이 적어 결국 더 빠름, 최고의 코딩 모델
- 팀 공유 CLAUDE.md 관리: git에 체크인, 팀 전체가 주 수회 업데이트, Claude가 잘못하면 즉시 추가
- GitHub Action으로 PR에 @.claude 태그: 코드 리뷰 중 CLAUDE.md 업데이트를 PR 작업의 일부로 자동화할 수 있음. CLAUDE.md는 계속 업데이트되어야 하며 너무 길지 않게(Boris Cherny의 경우 2500 tokens 정도) 유지하는 것이 바람직
- PostToolUse 후으로 자동 코드 포매팅: CI 포맷 에러 방지, Claude 생성 코드의 마지막 10% 처리. 테스트, 빌드, 린트 등이 성공적이었는지 확인
- 검증 루프 구축이 가장 중요: Claude가 스스로 결과를 검증할 수 있어야 함. TDD, 브라우저 테스트 (Playwright) 등을 포함

- 서버에이전트 활용 : code-simplifier, verify-app 등 반복되는 작업을 자동화. 그러나 무조건 많이 늘리는 것이 좋은 일은 아니며, '다른 관점으로 작동해야 하는' 경우에만 서버에이전트를 추가하는 방식이 권장됨
- 병렬 5개 인스턴스 운영: 터미널 탭 1~5번에 각각 Claude Code 작업을 실행하고, 시스템 알림으로 입력 필요 시 즉시 파악. 가급적 각각 독립된 작업이 아니면 향후 소스 통합에서 어려움이 있을 수 있음
- 웹 + 모바일 병행: claude.ai/code에서 추가로 5~10개 세션 동시 운영, iOS 앱에서 아침마다 세션 시작 후 체크인하여 작업
- 슬래시 커맨드로 반복 워크플로우 자동화: /commit-push-pr 등 하루 수십 번 사용, .claude/commands/에 git 체크인. '모든 걸 자동화해야 한다'가 아니라 효율을 위해 자주 쓰는 명령어만 자동화하는 것
- 권한 설정 세밀 관리 : --dangerously-skip-permissions 대신 /permissions로 안전한 명령어만 사전 허용하도록 설정
- MCP 서버로 외부 툴 연동: Slack, BigQuery, Sentry 등을 Claude가 직접 사용, .mcp.json으로 팀 공유

- 복잡한 셋업이 꼭 필수는 아님, 자신의 workflow에 맞는 셋업이 필요
- Boris Cherny는 Anthropic 내부자이며, 사실상 무제한의 토큰을 사용할 수 있음(비용 이슈 없음): 실제로는 비용 효율을 고려하여 Pro를 쓰는 사용자도 많으며, Max 5x나 Max 20x도 무제한은 아님
- CLAUDE.md의 내용이 늘어나거나, Skill, Command 등을 추가하면 토큰 소모량은 증가할 수 밖에 없으므로 문서들은 꼭 필요한 내용으로 정리하고, Skill과 Command도 사용하는 것만 설치
- 자주 사용하는 Skill과 Command 위주로 설치해두면 긴 프롬프트 입력을 대체할 수 있으므로 편리: 효율이 높아지고, 팀내에 공유하면 일관된 방식으로 작업하는데 도움이 됨
- 복수개 agent 구동은 효율을 올릴 수 있으나, 토큰을 매우 빠르게 소모하는 편(5시간 제한과 주간 제한에 유의)
- Sonnet 4.6 정도면 복잡하지 않은 일반 코드 작업에는 충분하며, 대규모 코드베이스 변경이나 설계에는 Opus가 나은 결과를 보여줌. Haiku는 간단한 작업에만 사용하는 것을 추천
- 가장 중요한 건 검증으로, TDD나 Playwright 등을 적극 활용하여 Claude Code가 스스로의 작업을 확인하고 검증할 수 있게 셋업해야 함

- 여러 AI CLI 에이전트(Claude Code, Codex CLI, Gemini CLI)를 동시에 운용하는 협업 구조
- 각 에이전트는 독립적으로 실행되며, 동일한 코드베이스를 서로 다른 관점에서 리뷰·작업
- tmux를 활용해 터미널을 multi pane으로 분할, 에이전트별 독립 세션 운영
- 에이전트마다 별도 계정 인증 필요 (Anthropic / OpenAI / Google OAuth)
- 각 실행 환경은 폴더 단위로 독립 구성되어 충돌 없이 병렬 실행 가능
- 예를 들어 Claude Code가 계획을 세우고, 스크립트를 통해 Codex나 Gemini에게 프롬프트로 작업을 전달한 후 결과를 받아보고 진행여부를 결정할 수 있음. 이 때 Claude Code는 Orchestrator 역할이며, Codex나 Gemini는 Sub Agent로서 기능함
- 각 모델의 독립적 관점 활용으로 단일 모델 대비 편향 감소: 각 모델의 장점을 십분 활용할 수 있음
- 단순 작업은 저렴한 모델에 위임해 API 비용 합리적 운용 가능